

Document Production (C.0)

Document Print Technical User Guide

The Document Production Document Print Technical User Guide provides a high-level overview of the Document Print process and describes how it can be interfaced with a Web application.

Revision date - June 10, 2008



© 2008 Computer Sciences Corporation. El Segundo, California. All rights reserved. No part of this publication may be reproduced by any means without written permission from Computer Sciences Corporation. Printed in U.S.A. All questions regarding this documentation should be routed through customer assistance, Blythewood, SC, 803-333-6200 or email pcssupport@csc.com.

Table of Contents

Chapter 1: Overview.....	4
Chapter 2: User Workflow.....	5
On-Demand Document	5
HTML Representation of the Document.....	6
Proof Page.....	7
Chapter 3: Technical Processing Details.....	10
Sequence of Events	10
Application Architecture - High Level.....	13
Application Architecture - Detail Level.....	15
Overview of the Web Service Layer.....	16
Overview of the Business Logic Layer	19
Application Directory Structure	21
Chapter 4: Web Methods Used by Media Management	23
Implementation Instruction	23
Chapter 5: WIP Forms for Immediate Print Transaction	26
Workflow	28
Implementation Instruction	31
Chapter 6: Other Components of Document Print.....	32
Document Print Initialization File - DOCPRINT.INI.....	32
Configuring DOCPRINT.INI File for Multiple Resource Libraries	33
FSIUSER and FSISYS Initialization Files.....	35
Document Print HOOK Functions.....	39
PrintHook Function.....	39
ArchiveHook Function.....	39
The HTML Print Driver	41
JavaScript	42
CSCCImportDPXMLFile Rule.....	45
WIP Purge Program.....	46
Chapter 7: Additional Features of Document Print.....	47
Document Print Clean Utility Job.....	47
Configuring the DPCleanUtil.INI File.....	47
Installing DPCleanUtil.....	50
Uninstalling DPCleanUtil.....	51

Multiple DPCleanUtil Utility Job running on the Same Machine	51
User Selectable Paragraphs.....	53
Modify Document Production	55
Modify Calling Application	57
Spell Checker.....	58
Modify Document Production	58
Modify Calling Application	59
Electronic Signature	60
User Workflow	60
Technical Processing	63
Designing the Document.....	64
HTML Print Driver	64
Storing Captured Signature on the Document Production Server.....	66
Embedding the Signature's Image into the Document.....	67
Implementation.....	68
Modifying Document Production.....	68
Forms and Variable Fields List Function	71
Implementing the Forms and Variable Fields List Function.....	73
Modify Document Production.....	73
Modify Calling Application.....	73
Exclude HTML Forms with No Manual Entry Fields from Rendering in the Browser.....	75
Implementation.....	75
Chapter 8: How to Manually Install Document Print.....	77
Install Document Print as a One Server Configuration.....	79
Install Document Print as a Two Server Configuration	82
Application Server Installation	82
Web Server Installation.....	83
Chapter 9: Document Change Log	84
Appendix A: Document Print Web Service Interface Definition	85
Appendix B: Media Management Web Method Interface Definition	100

Chapter 1: Overview

Document Print is a Web-based solution that allows users to manually initiate the printing of documents. It is an on-demand process that accepts a print-ready data file, merges data with the form generated, and displays the result in a browser over the intranet and Internet. Document Print allows you to view, modify data, and print the form through immediate print or batch print and archive.

Important: *Document Print provides a Web service that produces and archives documents. To be useful, Document Print must be interfaced with a Web application that is responsible for the user interface.*

Chapter 2: User Workflow

This section describes the workflow for a typical Document Print transaction using CSC's Exceed Browser application as an example. Other Web Applications could emulate this workflow or modify it as required. For more detailed information about the Exceed browser application, refer to Exceed documentation on Document Print.

On-Demand Document

To produce an on-demand document, click on a hyperlink or button that produces a page similar to the one below. This page lists all of the documents that you can create. The list, which is stored in a database maintained by the Web application, may be keyed on transaction, state, and line of business (product).

The screenshot displays a web application interface for selecting documents. It features several input fields and buttons for filtering and searching. The 'State' dropdown is set to 'Kansas'. The 'Line of Business' dropdown is empty. The 'Document Name' and 'Document ID' fields are also empty. A 'Search' button is located to the right of the 'Line of Business' field, and a 'Reset' button is to the right of the 'Document ID' field. Below these fields is a checkbox labeled 'Filter Document List:'. A text area labeled 'Document Name/Document ID:' contains a list of document types: 'CLAIM DENIAL LETTER CWSDENLTR', 'CLAIM LAWYER REP LETTER CWSLAWREP', 'CLAIM MEDICAL REQUEST CWSMEDREQ', and 'CLAIM REP LETTER CWSREPLTR'. At the bottom, there is a large empty text area labeled 'Documents Printed This Session:' and a 'Select' button.

Document Name/Document ID
CLAIM DENIAL LETTER CWSDENLTR
CLAIM LAWYER REP LETTER CWSLAWREP
CLAIM MEDICAL REQUEST CWSMEDREQ
CLAIM REP LETTER CWSREPLTR

Figure 1. All Documents Listed

Proceed by selecting one of the documents from the document list and click **Select**.

HTML Representation of the Document

The next page that may be displayed is an HTML representation of the document, at which point you can edit fields or enter free-form text. The Document Designer specifies which fields can be edited and enforces some specific data types (currently alphabetic, numeric, and date). If the Web application knows that a particular form contains no enterable fields, the Web application could skip this screen completely and proceed immediately to the proof page.

CSC **CSC Personal Benefits Insurance Center**
P. O. Box Ten, Columbia, SC 29202-0010
1-800-333-4000

March 14, 2005

Tanya Gardner 124A
155 Cherry Lane
Gilbert, KS 66001

RE: Policy Number: APVTANYA124A
Insured: Tanya Gardner 124A
Date of Loss: 09/01/2003

We have received your claim and have had an opportunity to review the loss information and inspect your property. We must respectfully decline payment of your claim for the following reason(s):

- ☐ 1. No damage to the property was noted by our Claims Representative.
- ☒ 2. The amount of damage assessed does not exceed your policy deductible.
- ☐ 3. Other

Should you have additional information concerning this claim, please contact us.

Sincerely,

Sandra M. Orr
Claims Representative

[Document Index](#) | [Exit](#) | [Help](#)

Continue

Figure 2. HTML Representation of the Document

When you have finished reviewing the document and modifying any field values, click **Continue** to proceed to the final proof page.

Proof Page

The proof page displays the document using the Adobe Acrobat Reader browser plug-in or other viewers. The proof version of the document is an exact replica of what the document would look like when printed, as opposed to the HTML representation, whose appearance is restricted by the limitations of HTML.

CSC **CSC Personal Benefits Insurance Center**
P. O. Box Ten, Columbia, SC 29202-0010
1-800-333-4000

March 14, 2005

Tanya Gardner124A
155 Cherry Lane
Gilbert, KS 66001

RE: Policy Number: APVTANYA124A
Insured: Tanya Gardner124A
Date of Loss: 09/01/2003

We have received your claim and have had an opportunity to review the loss information and inspect your property. We must respectfully decline payment of your claim for the following reason(s):

() 1. No damage to the property was noted by our Claims Representative.
(X) 2. The amount of damage assessed does not exceed your policy deductible.
() 3. Other _____

Should you have additional information concerning this claim, please contact us.

Sincerely,

Sandra M. Orr
Claims Representative

1 of 1 8.5 x 11 in

Immediate Print Batch Print

[Back](#) | [Document Index](#) | [Exit](#) | [Help](#)

Figure 3. Proof Page

The proof page has two buttons to support Immediate and Batch Print.

Clicking **Immediate Print** presents Adobe Acrobat Reader Print dialog, which can be used to print the document to a local or network printer. You will then be presented with a dialog box asking if the document printed successfully. Select **YES** to archive the document. If you select **NO**, nothing will happen. You can print the document by using the Adobe print function directly, but the document would not be archived.

If you click **Batch Print**, the document will be saved in the Document Production Work In Process (WIP) system, where it can be merged into the next high volume Document Production batch run. This saves you from having to handle the printed documents.

Note: *Document Print does not provide any screens or Web pages. All of the pages shown above are examples from CSC's Exceed system. If you want to use Document Print with another application, you must build similar pages.*

Chapter 3: Technical Processing Details

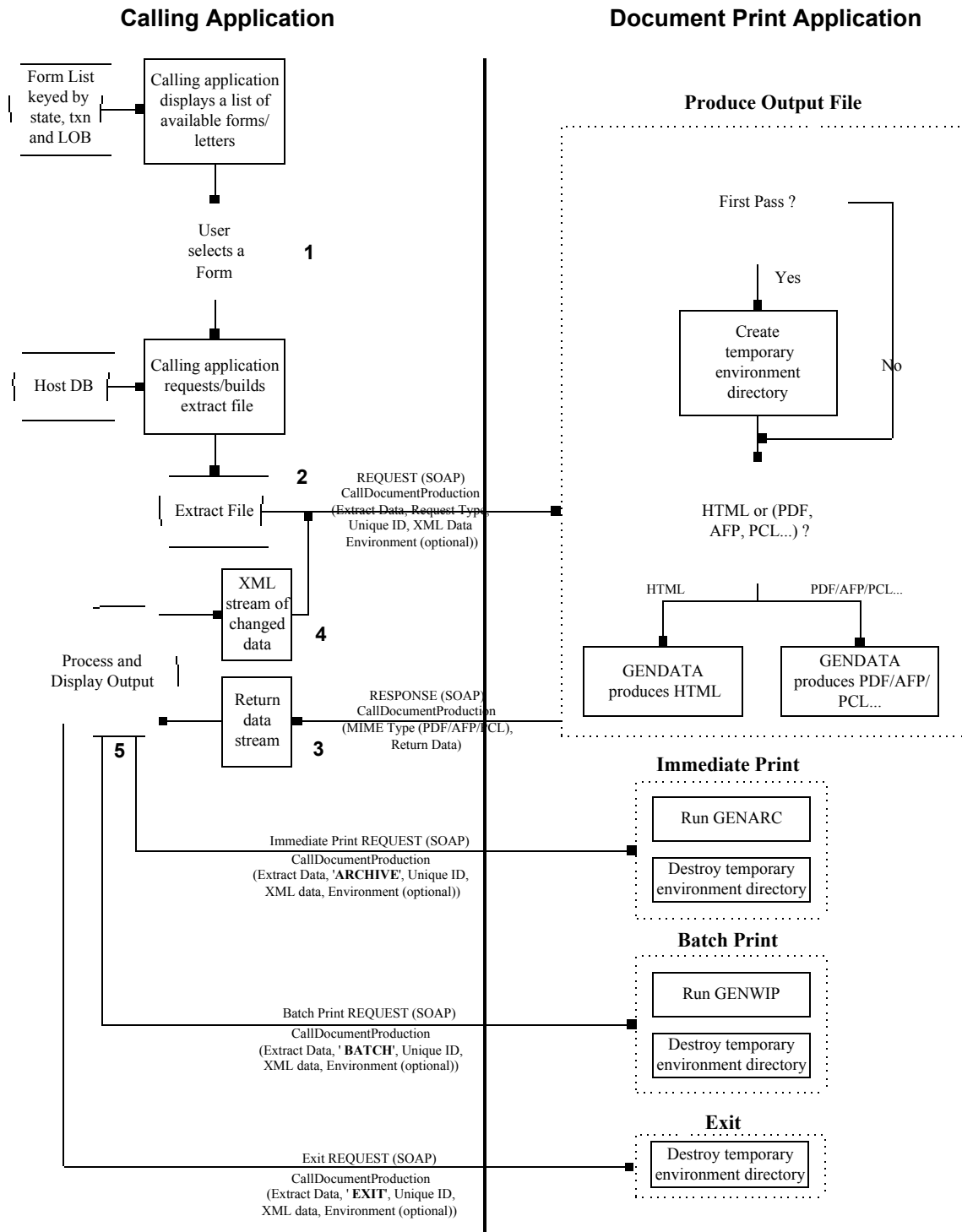
Sequence of Events

Document Print is a Web service application that is designed to receive a request from the calling application and, based on the request sent, process one of several Document Production components, producing a response to the calling application. This section describes the sequence of events for CSC's Exceed application. Other calling applications can use this process as a model.

- 1 A function within the calling application (for example, Exceed) is selected indicating that an on-demand document is to be produced. A candidate list of forms is presented and one of the forms is selected.
- 2 The calling application generates an extract file containing the data to appear on the document. This may require the calling application to call its host application (for example, on a mainframe or AS/400). The calling application invokes Document Print, passing to it the extract file, an XML list of variable fields (not applicable for HTML requests), and a request type indicating the type of output stream to produce (HTML, PDF, or any other valid Document Production print stream). The message is passed using Simple Object Access Protocol (SOAP).
- 3 Document Print returns the document either in HTML format, if customization is allowed, or as a PDF if print preview is desired. If the data returned is an HTML document, the Web page will have a Continue button. If the file is a PDF document, the Web page will have Immediate Print and Batch Print buttons.
- 4 If the **Continue** button is clicked, an XML list containing variable fields on the HTML form is created which Document Production will use to overwrite the default values from the extract file. The calling

application then calls Document Print for the second time to create a PDF file. Step three is then repeated.

- 5 If the **Immediate Print** or **Batch Print** button is clicked, a request is issued to Document Print to archive or batch queue the transaction.
- 6 Document Print returns a code indicating success or failure of the request and an appropriate message is displayed indicating that the transaction is complete or in error.

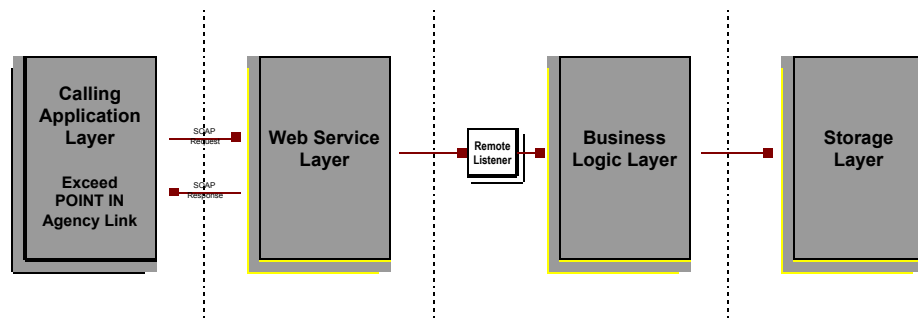


Application Architecture - High Level

Document Print is a .NET Web base application that is organized in four layers:

- Calling Application Layer
- Web Service Layer
- Business Logic Layer
- Storage Layer

Following is a high-level illustration of the Document Print architecture:



The calling application layer refers to an Enterprise system of CSC, such as Exceed, POINT IN, or Agency Link. This layer is responsible for invoking the Web Services provided by the Web Service layer, passing them the appropriate information such as extract data, and processing the returned responses.

Communication between this layer and the Web Service layer is handled using the SOAP protocol. SOAP provides an open, extensible way for applications to communicate using XML and HTTP to send messages over the Web regardless of what operating system, object model, or language a particular application may use. Additional information about SOAP can be found at Microsoft's Web site <http://www.microsoft.com>.

Further details about the calling application layer are not in the scope of this document. For additional information, refer to the CSC system documentation that is interfacing with Document Print.

The Web service layer contains a number of Web methods that allows the calling application to call to perform specific functions. It also provides a higher-level interface to a set of lower-level interfaces in the business logic layer. It is responsible for manipulating

incoming requests as necessary, sending them on to the appropriate business objects, handling the responses returned from the business layer, and passing the results back to the calling application.

The business logic layer exposes a number of classes that can be called by the Web Service layer. It is responsible for processing the incoming request from the Web service layer.

The Web service layer and business logic layer can reside on the same server or on separate servers. If the Web Service layer is on a different server, a remote listener is used by the server containing the business logic layer. The listener simply listens to a specified port and directs any incoming connection requests to the appropriate business class.

The storage layer is responsible for the physical storage of archived and WIP documents. This layer is simply an SQL server database containing the WIP and Archive tables. Skywire's base system defaults to a flat file database (DBASE IV). However, Document Print must be configured to use an SQL database for performance reasons as well as the ability to handle concurrent GENWIP or GENARC runs. GENWIP and GENARC are Skywire's batch processing modules used by Document Print to send documents to WIP and Archive.

Application Architecture - Detail Level

This section gives a detailed overview of the Web Service layer and business logic layer of Document Print. It will discuss the different Web Methods that are contained in the Web Service layer and the different classes that make up the business logic layer. The storage layer is not shown because it only consists of an SQL Server database.

Below is a detailed illustration of the Document Print architecture.

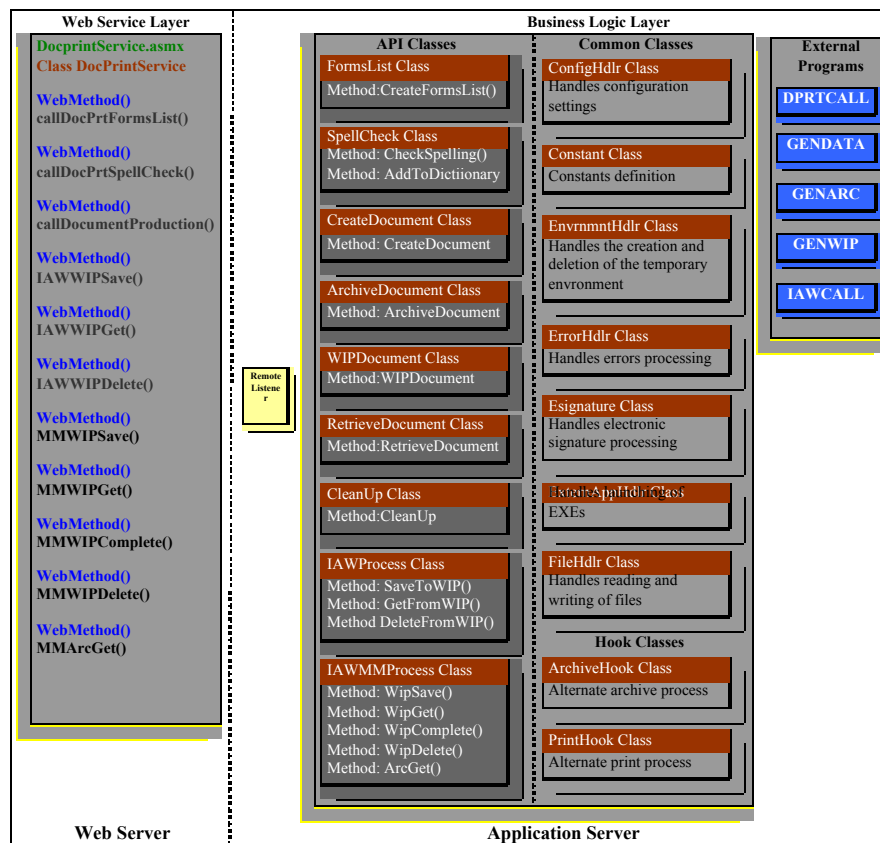


Figure 4. Document Print Architecture

Overview of the Web Service Layer

The Web Service layer contains several Web methods for use in creating applications. These Web methods permit a calling application to send data and retrieve a result created by the business logic layer.

The table below is intended to provide high-level information about each Web Method in the Web Service layer. For detailed information about the various call parameters and the data to be passed, see the appendix, [Document Print Web Service Interface Definition](#).

WEBSERVICEAPI

Web Method	Function
callDocPrtFormsList	Generates a list of forms using the Form Set Definition file. Generates a list of variable fields on a form.
callDocPrtSpellCheck	Checks the spelling of input fields on the HTML document. Adds custom words to the dictionary.
callDocumentProduction	Creates a document in HTML or printable format. Sends a document to archive. Sends a document to WIP for batch processing.
Web Methods used by Exceed	
IAWWIPSave	Saves a document to WIP for later retrieval.
IAWWIPGet	Retrieves a document from WIP that was saved using the IAWWIPSave.
IAWWIPDelete	Deletes a document from WIP that was saved using the IAWWIPSave.
Web Methods used by Media Management	
MMWIPSave	Saves the document to WIP
MMWIPGet	Retrieves the document from WIP
MMWIPComplete	Completes the document in WIP either by archiving the document or marking the document for batch printing
MMWIPDelete	Delete the document from WIP
MMARCGet	Retrieves a PDF version of the document from archive

The *callDocumentProduction* Web method is the main API for Document Print. It allows the calling application to produce on-demand documents and store them in the archive or WIP database.

This API requires one of several request types to indicate what function the calling application wants to perform. The table [Request Types](#) lists the available request types and what action is performed for each.

REQUEST TYPES

Request Type	Action
HTML	Produces a document in HTML allowing the user to enter or modify variable field data. This request type uses the FSIUSER.HTM file configured to produce HTML documents.
PRINT	Produces a printable version of the document. Request type uses the FSIUSER.INI file configured to produce one of the following supported formats: AFP, PCL, PDF, PST, or RTF.
PDF, PCL, AFP, PST, or RTF	Same as PRINT request type but uses a FSIUSER file with the extension being equal to the request type. For example, a PDF request type will use the FSIUSER.PDF file configured to produce PDF documents.
RETRIEVE	Retrieves the document that was produced by a previous PRINT request for faxing or emailing.
ARCHIVE	Archives the completed document.
BATCH	Sends the document to WIP with a status code of "B." This queues the document in WIP for batch processing and will be included in the next high-volume batch cycle.
EXIT	Deletes the temporary directory created for the current transaction.
PNT_WIP	Creates a document and sends it to WIP with a status code of "W." The document is not returned but can be retrieved using Media Management.
ARONLY_???	Creates and archive a document but doesn't return a print file
ARCRET_???	Creates and archive a document and returns a print file
WIPONLY_???	Creates and sends document to WIP but doesn't return a print file
Note: ??? indicates file extension of the FSIUSER file to use (i.e. ARCRET_PDF=FSIUSER.PDF)	

Overview of the Business Logic Layer

The business logic layer consists of the following sub-layers:

- API Classes
- Common Classes
- External Executables

The API classes expose the lower-level interfaces of the business logic layer to the Web Services. Each class has a specific function to perform and is responsible for executing and returning the appropriate result back to the calling Web Service. The table below provides a summary for each API class.

BUSINESS LOGIC API CLASS

API Class	Called By	Function
FormsList	callDocPrtFormsList	Returns an XML list of forms or variable fields.
SpellCheck	callDocPrtSpellCheck	Returns an XML list of misspelled words on the HTML document. Add custom words to the dictionary.
CreateDocument	callDocumentProduction	Creates a document in HTML format or print version format.
ArchiveDocument	callDocumentProduction	Sends a document to archive.
WIPDocument	callDocumentProduction	Sends a document to WIP to be processed by the next batch cycle.
RetrieveDocument	callDocumentProduction	Retrieves a document that has already been created from a previous PRINT request.
CleanUp	callDocumentProduction	Deletes the temporary directory.
IAWProcess	IAWWIPSave IAWWIPGet IAWWIPDelete	Saves a document to WIP. Retrieves a document from WIP. Deletes a document from WIP.
IAWMMPProcess	WIPSave WIPGet WIPComplete WIPDelete ArcGet	Saves a document to WIP Retrieves a document from WIP Completes a document in WIP Deletes a document in WIP Retrieves a document from archive

The common classes provide a set of common functions that are used by the business logic API classes to carry out a request. For example, the **FileHdlr** class contains a number of functions that handle the reading and writing of files. The table [Common Class](#) provides a summary for each common class.

COMMON CLASS

Common Class	Function
ConfigHdlr	Handles the loading of the DOCPRINT.INI file and sets property values used by the program.
Constant	Holds all constants definition.
EnvrnmntHdlr	Handles the creation and deletion of the temporary environment.
ErrorHdlr	Handles error processing.
Esignature	Handles the processing of electronic signatures.
ExternAppHdlr	Handles the launching of the external executables.
FileHdlr	Handles the reading and writing of files.

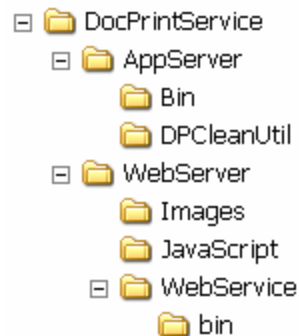
The external executables consist of Skywire's batch processing modules and modules developed by CSC. The table [External Executable](#) provides a summary of each executable.

EXTERNAL EXECUTABLE

Executable	Function
DPRTCALL	A CSC value add module that: • performs the main logic for generating a list of forms or variable fields • Spell checks variable data gathered from the HTML document • Adds custom words to the dictionary
IAWCALL	A CSC value add module used by the IAWProcess class to save, retrieve, or delete a document from WIP.
GENDATA	Skywire's module that is responsible for triggering a document, applying processing rules, and flooding the document with data from the extract file.
GENARC	Skywire's module that sends the document to archive.
GENWIP	Skywire's module that sends the document to WIP.

Application Directory Structure

The figure below shows the directory structure for the Document Print application.

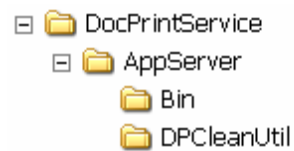


The *DocPrintService* folder contains the Document Print application and is installed under the target directory specified during the installation (for example, c:\DocProd). The *AppServer* subdirectory contains the components for the application server, and the *WebServer* subdirectory contains the components for the Web server. In a one server configuration, both the *AppServer* and *WebServer* subdirectories will reside on the same server. In a two server configuration, the *AppServer* and *WebServer* subdirectories will reside on different servers as illustrated below:

Web Server



Application Server



The table below provides a description for each folder:

APPLICATION STRUCTURE

Folder		Description
<u>AppServer</u>	Bin	Contains business logic assembly; Document Print configuration file (DOCPRINT.INI); and remote listener assembly and configuration file.
	DPCleanUtil	Contains the Document Print clean utility assembly and configuration file.
<u>WebServer</u>	Images	Contains document logo files that have been converted to GIF, JPG, or BMP format so they can be displayed on the HTML document.
	WebService	Contains the Web service file (asmx).
	WebService\bin	Contains Web Service layer assemblies.
	Javascripts	Contains Javascript files that are used by the HTML document.

Chapter 4: Web Methods Used by Media Management

In release C.0, new Web methods were added to the Document Print Web service for Media Management to access functions contained in the IAW module. This allows isolation of the Document Production archive and WIP data on a separate data server without any LAN connections required on the Media Management server. It also eliminates duplication of Document Production settings in the Media Management settings files.

Below is a high-level overview of the new Web methods added to Document Print for Media Management. For detailed information about the various call parameters and the data to be passed, see the appendix, Media Management Web Method Interface Definition.

Web Method	Description
MMWIPSAVE	Saves a document to the Document Production WIP database
MMWIPGET	Retrieves a document from the Document Production WIP database
MMWIPCOMPLETE	Completes a document in the Document Production WIP database either by archiving the document or marking the document for batch printing.
MMWIPDELETE	Deletes a document from the Document Production WIP database
MMARCGET	Retrieves a document from the Document Production Archive database

Implementation Instruction

Before Media Management can use the new Web methods in Document Print, the following updates to the Document Production environment are required.

- 1 Add the following entry to the WIP.DFD file located in DEFLIB. The IAWMMTAG key is used by IAW to retrieve documents stored in WIP.

```
< KEYS >
KEYNAME      = IAWMMTAG
< KEY:IAWMMTAG >
EXPRESSION = KEY1+KEY2+KEYID+FORMSETID
FIELDLIST = KEY1,KEY2,KEYID,FORMSETID
```

- 2 Add the following group to your Document Print FSISYS.INI file. The options listed under this group below are for illustration purposes and may be different from your settings.

```
< AfeWip2ArchiveRecord >
Company      = KEY1
Lob          = KEY2
PolicyNum    = KEYID
RunDate      = APPDATA
TrnType      = TRANCODE
LastName     = lastname
EmailAddr    = EmailAddr
mconum       = mconum
MVKey        = MVKey
Issue        = issuecode
EffDate      = EffDate
RecipList    = RecipList
```

- 3 Add the following group to your Document Print FSISYS.INI file

```
< STATUS_CD >
BATCHPRINT = B
ARCHIVE     = AR
```


Chapter 5: WIP Forms for Immediate Print Transaction

The immediate print transaction in Document Print is a process that generates on-demand policy declarations. Prior to the C.0 release, this process only generated the proof version of the document. However, if the policy declaration contained any WIP forms, forms containing fields that require entry from the user, the user was unable to complete the document before sending the document to archive.

In release C.0, Document Print was modified to handle generating policy declarations containing WIP forms. Whenever a policy declaration is triggered, Document Print will determine if the document contains any WIP forms by checking the document for fields having the POWTYPE or KICK2WIP field rule and a value of "Y" in the field's required operator field. If these criteria are met, the document will be generated in HTML format, so the user can complete the document.

In addition, only forms containing WIP fields are returned in HTML format and not the entire policy declaration. This allows the user to only view the forms that need to be completed. When the proof version is requested, the complete policy declaration is produced, including the updated WIP fields.

The following new request types were created for the immediate print process.

Request Type	Description
IMP	Instructs Document Print to check the document for WIP fields
IMP_PDF	Instructs Document Print to produce the document in PDF format after the WIP forms have been completed
IMP_PCL	Instructs Document Print to produce the document in PCL format after the WIP forms have been completed. Used by customers who are not licensed for the PDF print driver

Request Type	Description
IMP_WIP	Instructs Document Print to send the document to WIP with status code of "W"

Note: The value of IMP will instruct Document Print to use the FSIUSER.IMP file. This FSIUSER file is configured to produce policy declarations.

In addition a new custom rule called CSCCCheck4WipForms was created to check for WIP fields on the policy declaration that was triggered. If the rule determines that the document has WIP fields, the print type in the internally loaded INI file is changed to HTM. This causes the HTML print driver to be executed.

Workflow

The workflow described below is for a typical immediate print transaction in a POINT-IN application. It also shows what values are sent in the request and what values are returned in the response for the SOAP call.

WebMethod:

Public Function callDocumentProduction

(ByVal strReqType As String, _
ByVal strUniqueID As String, _
ByVal strEnvrnmnt As String, _
ByRef strRetMIMEType As String)
As Integer

- 1 User initiates a request to generate a Policy Declaration.

Request	
Field Value	Description
strReqType: IMP strUniqueID: 12345 strEnvrnmnt: POINTDMS strRetMIMEType:	Request type IMP instructs Document Print to check the document for any WIP fields
Response	
Return Code: 0 or -1 strRetMIMEType: htm or pdf	Document is returned in HTML or PDF format. The field strRetMIMEType indicates the type of document returned.

- 2 If the document contained no WIP fields, the proof version is displayed and the user will have the option of archiving the document.
- 3 If the document contained WIP fields, the HTML document is displayed and the user will have the option of either saving the document to WIP or previewing the proof version.

If the user saves to document to WIP:

Request	
Field Value	Description
strReqType: IMP_WIP strUniqueId: 12345 strEnvrnmnt: POINTDMS strRetMIMEType:	Request type IMP_WIP instructs Document Print to send the document to WIP. Document is stored in WIP with a status code of "W" so it can be later retrieved using Media Management.
Response	
Return Code: 0 or -1	Return code of 0 or -1 to indicate success or failure.

If the user requests a proof version:

Request	
Field Value	Description
sstrReqType: IMP_PDF strUniqueId: 12345 strEnvrnmnt: POINTDMS strRetMIMEType:	Request type IMP_PDF instructs Document Print to produce the document in PDF format after the WIP forms have been completed
Response	
Return Code: 0 or -1 strRetMIMEType: pdf	Field strRetMIMEType indicates the type of document returned

- 4 On the preview screen the user has the option of editing the document again, archiving the document, or sending the document to WIP for batch print.

If the user request to edit the document again:

Request	
Field Value	Description
strReqType: IMP strUniqueId: 12345 strEnvrnmnt: POINTDMS strRetMIMEType:	Request type IMP is sent to Document Print so the document can be returned in HTML format again. Same call as the initial call.
Response	
Return Code: 0 or -1 strRetMIMEType: htm	Field strRetMIMEType indicates the type of document returned.

If the user request to archive the document:

Request	
Field Value	Description
strReqType: ARCHIVE strUniqueId: 12345 strEnvrnmnt: POINTDMS strRetMIMEType:	Archives the document.
Response	
Return Code: 0 or -1	0 or -1 indicates success or failure

If the user request to send the document to WIP for batch printing:

Request	
Field Value	Description
strReqType: BATCH strUniqueId: 12345 strEnvrnmnt: POINTDMS strRetMIMEType:	The document is sent to WIP with a status of "B" for batch printing.
Response	
Return Code: 0 or -1	0 or -1 indicates success or failure

Implementation Instruction

- 1 Create an AFGDEC.JOB file containing the necessary rules, global variables, etc. to produce policy declaration documents. This file also needs to be configured for single step processing.
- 2 Add the following custom rule to the AFGDEC.JOB created in step one. This rule must come after the PaginateAndPropagate rule.

```
;CSCCCheck4WipForms;2;;
```

- 3 Add the following option to the FSISYS.INI file used by Document Print

```
< PRTTYPE:HTM >
::
LimitFormsByPowType = Yes
```

Add the new FSIUSER.IMP file containing the following options to the Document Print resource library:

```
[ MASTERRESOURCE ]
FORMDEF = FORM.DAT

[ DATA ]
AFGJOBFILE = AFGDEC.JDT

[ ENVIRONMENT ]
FSISYSINI = .\FSISYS.INI

[ PRINTER ]
PRTTYPE = PDF

[ MANPRINT ]
PORT = PRINT\MANUAL.PDF

[ STATUS_CD ]
WIP = W

[ PRTTYPE:HTM ]
PrintMultiRecip = No
```

Note: *PrintMultiRecip indicates whether or not to produce a copy for each recipient. If option is set to "No", only one recipient copy is produced. If set to "Yes" all recipient copies are produced.*

Chapter 6: Other Components of Document Print

The following sections discuss other components that make up Document Print.

Document Print Initialization File - DOCPRINT.INI

Document Print uses an initialization file to set environment parameters and enable and disable certain features in the system. This initialization file is called DOCPRINT.INI and is located in the BIN directory under the AppServer folder.

The following table gives a brief description for each option in this file.

DOCPRINT.INI FILE

Group	Option	Description
Environment	MRL_LOC	Location of the Document Print resource library (full path required)
	DLL_LOC	Location of the Document Production DLL library (full path required)
	DPXMLFILE	Name of XML file containing the input fields on the HTML document
	EXTRFILE	The name to save the extract file as

Group	Option	Description
	RUNMODE	Instructs the system on how to run the GEN programs. A runmode of: 1 - GENTRN, GENDATA, and GENPRINT run in one step (single step processing) 2 - GENTRN and GENDATA run in one step and GENPRINT as a separate step 3 - GENTRN, GENDATA, and GENPRINT run in separate steps
	AUTOARC	Determines whether to run GENARC or not on an ARCHIVE request. Y—GENARC is run N—GENARC is not run
	PRTTYPE	MIME type of proof document (for example, PDF)
	DELETE_TEMP	Determines whether or not to delete the temporary directory. Y—Delete temporary directory N—Do not delete temporary directory
	PROCESS_MAXTIME	Controls the amount of time a process can run (for example, GENDAW32). Time is in seconds.

Configuring DOCPRINT.INI File for Multiple Resource Libraries

The DOCPRINT.INI file can be configured to have multiple environments sharing the same Document Print application. For example, the DOCPRINT.INI file that is installed with the base system is configured for multiple Document Print resource libraries.

To have multiple environments configured in your INI file you must suffix each ENVIRONMENT group with a name separated by a colon. Using the base INI file for illustration, this file contains two ENVIRONMENT groups with each group configured to their respective resource library.

```
[ ENVIRONMENT:EXCEED ]
MRL_LOC          = D:\DocProd\MSTRRES\DOCPRINT\EXCEED
DLL_LOC          = D:\DocProd\DLL
DPXMLFile        = XmlFile.Xml
EXTRFILE         = EXTRFILE.DAT
RUNMODE          = 1
AUTOARC          = Y
PRTTYPE          = pdf
ARCHIVE          = N
PRINT            = N
DELETE_TEMP      = Y
PROCESS_MAXWAIT  = 300

[ ENVIRONMENT:POINT ]
MRL_LOC          = D:\DocProd\MSTRRES\DOCPRINT\POINT
DLL_LOC          = D:\DocProd\DLL
DPXMLFile        = XmlFile.Xml
EXTRFILE         = EXTRFILE.DAT
RUNMODE          = 1
AUTOARC          = Y
PRTTYPE          = pdf
ARCHIVE          = N
PRINT            = N
DELETE_TEMP      = N
PROCESS_MAXWAIT  = 200
```

The calling application is responsible for telling Document Print which environment to use by passing the name of the environment in the environment parameter for each Web Service API. Document Print appends this value, prefixed by a colon, to the string ENVIRONMENT and the INI file is searched for a matching group name. Document Print then uses the settings listed under that group name for processing.

FSIUSER and FSISYS Initialization Files

In order for Document Print to run the GEN programs (GENDATA, GENWIP, and GENARC) to produce different types of document and send the document to WIP or archive, FSIUSER and FSISYS initialization files are needed. The FSISYS file contains most of the settings required to run the GEN programs, while the FSIUSER file will mainly contain settings overriding settings in the FSISYS file. For example, the FSIUSER.HTM file will contain an override of the PRTTYPE setting specified in the FSISYS file to produce HTML documents.

Depending on the requirements, many different FSIUSER files can be contained in a Document Print environment. For example, the sample POINTDMS folder installed under the DOCPRINT folder has several FSIUSER files with each serving a different purpose as described below.

FSIUSER.HTM	Contains settings to produce HTML documents for manual forms
FSIUSER.INI	Contains settings to produce PDF documents
FSIUSER.IMP	Contains settings to produce Policy Declaration documents for Immediate Print transactions
FSIUSER.WIP	Contains settings to produce PDF documents for manual forms and set the status code to "W" for WIP documents.

Since there can be a number of initialization files configured for different purposes, only the FSIUSER.HTM and FSIUSER.INI will be explained here. Other INI files required for a particular function will be explained under the appropriate section.

The FSIUSER.HTM file is used for HTML request to produce a document in HTML format. It contains the following required entries to override the default settings for PRINTER and MANPRINT group defined in the FSISYS.INI file.

```
[ PRINTER ]  
PRTTYPE = HTM  
[ MANPRINT ]  
PORT = PRINT\MANUAL.HTM
```

The print type (PRTTYPE) must equal HTM to instruct the GEN programs to use the HTML print driver to produce the document in HTML format.

The FSISYS.INI file contains the following additional settings for the HTML print driver:

```
< PRTTYPE:HTM >
Device           = LPT1
DownloadFonts    = Yes, Enabled
Module           = HTMW32
PrintFunc        = HTMPrint
OutputFunc       = HTMOutput
ChangeFields     = Yes
AddScripts       = Yes
ImageType        = jpg
GifPath          = http://localhost/DocPrintService/Images/
JSPath           = http://localhost/DocPrintService/JavaScript/htmscrip.js
ScriptFile       = http://localhost/DocPrintService/JavaScript/buildxml.js
ScriptFile       = http://localhost/DocPrintService/JavaScript/HTMESIGN.js
UseShadedAreaColor = No
UseBackgroundColor = Yes
DefaultFieldColor = B0E0E6
RequiredFieldColor = FFFF00
WaitOnLoad       = Message to be displayed until the document is loaded
OnLoadExecute    = init()
```

Where:

MODULE	DLL name of the HTML print driver.
PRINTFUNC	identifies the main print function that GENPRINT calls for all printing operations.
OUTPUTFUNC	identifies the function that is responsible for opening, writing to, and closing the print file.
CHANGFIELDS	specifies whether variable fields on the HTML document can be modified. If set to "No," all variable fields cannot be modified. If set to "Yes," all variable fields without the attribute "No User Edit" can be modified. Variable fields with the "No User Edit" attribute cannot be modified regardless of this value.
ADDSRIPTS	allows client-side JavaScript to be added to the HTML document to perform certain types of data entry validation and formatting. If set to "No," no reference to the JavaScript file is included. If set to "Yes," a reference to the JavaScript file is included.
IMAGETYPE	identifies the image file type that will be used for displaying in the browser.
GFPATH	specifies where the document's image files are located.
JSPATH	specifies the path and filename of the external JavaScript file

SCRIPTFILE	allows additional links to JavaScript files to be included in the HTML document.
USEBACKGROUNDCOLOR	determines whether input fields will have a color background. Default is "Yes." Yes - Input fields will have a background color specified in DefaultFieldColor. No - Input fields containing data will not have a color background.
DEFAULTFIELDCOLOR	specifies the background color for input fields. Specify the color in HEX value (for example, EFEBDE). If UseBackgroundColor=Yes and no color is specified, the default color will be FAF0E6.
REQUIREDFIELDCOLOR	specifies the text color to use for required fields. If a color is not specified, the default color will be the same as the color specified for DefaultFieldColor. Specify the color in HEX value (for example, FFFF00).
USESHADEDCOLOR	determines if shaded area will have the color specified in the Image Editor. Default is "No." No - Shaded area will have no color (clear background). Yes - Shaded area will use color specified in the Image Editor.
WAITONLOAD	displays a temporary status message page with the supplied message, letting the user know that a large document large is still loading in the browser. The document is hidden from the user, preventing the user from entering any data and/or submitting the document before the document has completed loading.
ONLOADEXECUTE	allows a JavaScript function to be specified to execute during the BODY ONLOAD event. In the example given above, the function init() will be called after the document has been completely loaded in the browser. The FSIUSER.INI file is used for request types producing a print version of the document, such as PDF, PCL, or AFP, and request types for sending the document to WIP or archive. It has the same overrides as the FSIUSER.HTM file but is configured to produce the print version of the document (see below).

```
[ PRINTER ]  
  PRTTYPE = PDF  
  
[ MANPRINT ]  
  PORT = PRINT\MANUAL.PDF
```

Document Print HOOK Functions

The Document Print application provides two shell functions that can be customized to alter the process of Document Print.

PrintHook Function

The *PrintHook* shell function allows custom code to be added, invoking an external program to convert the print stream produced by a PRINT request to some other format. For example, after a document is created in AFP format and before it is passed back to the browser, the *PrintHook* function can be called to execute custom code that calls a third party software to convert the AFP print stream to PDF. Instead of the AFP document being returned, the PDF document is returned. This hook function was included for customers who are not licensed for Skywire's PDF print driver and who need an alternative method for producing PDF documents.

Steps to implement this process:

- 1 Add your custom code to the PrintHook.vb module located in:

SDK\DocPrintService\AppServer\BusinessLayer\Hooks

- 2 In the DOCPRINT.INI file, update the following setting under the ENVIRONMENT group:

```
PRTYPE = File type returned to the browser
          (PDF, AFP, etc.)
PRINT = Y (Use the Hook function)
```

ArchiveHook Function

The *ArchiveHook* shell function allows custom code to be added to invoke an alternative method for archiving documents other than using Skywire's archive system. For example, a customer may want to add custom code that will copy a completed document print file to a staging area where it will be uploaded to the mainframe for archiving.

Steps to implement this process:

- 1 Add your custom code to the ArchiveHook.vb module located in:

SDK\DocPrintService\AppServer\BusinessLayer\Hooks

- 2 In the DOCPRINT.INI file, update the following setting under the ENVIRONMENT group:

```
AUTOARC = N (Does not run GENARC)
ARCHIVE = Y (Use the Hook function)
```

The HTML Print Driver

The HTML print driver was created by CSC to produce triggered documents in HTML format, allowing you to change or enter missing data for input fields in the browser. It was designed for a single recipient processing a single transaction. For each object contained on the document, the HTML print driver will produce the appropriate HTML tag as described in the following table.

FORM OBJECT TO HTML TAG CONVERSION

Object	Converted To
Font	A STYLE tag. STYLE tags include attributes such as point size, font family, font style, etc. Each different font gets named as TIFD##### or VFID#####, where ##### is the font ID as found in the font cross reference file. The first two letters, TF or VF indicate static field or variable field fonts respectively.
Form	A FORM tag. The FORM tag includes a NAME attribute that has a value matching the form name specified in the FORM.DAT file.
Image (FAP)	A DIV tag. The DIV statement specifies where the image begins. The height is based on the actual FAP dimension. The DIV ID is the same name as the FAP name.
Logo	An IMG tag. In order to display logos in the browser, they will have to be converted into JPEG or GIF files. The IMG tag will contain a path pointing to the location of the JPEG/GIF files.
Static Text	A SPAN tag. Includes text label and text areas (also called paragraphs).
Lines/Boxes	An HR tag. Contains position, height, and width as specified on the FAP. The HTML print drive does not support boxes that are shaded. Therefore, all boxes are converted into simple boxes.
Variable Field	An INPUT tag. Fields that can be edited will have a color background. Fields that are mapped from the extract file and have the "No User Edit" attribute will be displayed in red.

JavaScript

Document Print uses client-side JavaScript to validate and format input field data on an HTML form. As an option, JavaScript functions can be added to an HTML document by setting the INI option *AddScripts* to “Yes.” These functions are stored in an external file for ease of customization.

For each Javascript file to be included in the HTML document, the HTML print driver includes a SCRIPT tag indicating where the JavaScript file is located. The path to the JavaScript file is controlled by Document Print's FSISYS.INI file option JSPATH.

The HTML print driver uses information stored in the FAP file when creating JavaScript for input fields. For example, the Image Editor converts a variable field's format to a code and places it in the FAP. The HTML print driver uses this code as the value in the function's format parameter.

The base system provides JavaScript validation and format functions for fields having data types of numeric, alphabetic, alphanumeric, multi-line text, and date.

The following functions are currently provided:

isNumeric(this, format)

This function is for fields defined as numeric. It validates data in the current field for all numbers. It also formats data based on format mask selected in the Image Editor. For example, if a user enters “124500” for a field on the HTML document and the format mask selected is “\$ZZZ,ZZ9.99,” the function will convert the data to “\$1,245.00.” This entry that the HTML print driver would create for the previous example:

```
IsNumeric(this, '$ZZZ,ZZ9.99')
```

isAlphabetic(this, format)

This function is for fields defined as alphabetic, alphanumeric, and uppercase alphabetic and alphanumeric. It validates and formats data based on the FAP code specified in the *format* parameter. FAP codes are internal codes used by the system and are based on the format mask selected in Image Editor.

This table lists FAP codes and the action performed on the field:

FORMAT CODE

Code	Action Performed
a	Field is validated to contain all alphabetic characters
A	Field is validated to contain all alphabetic characters and is converted to all uppercase
x	Field is validated to contain all alphanumeric characters
X	Field is validated to contain all alphanumeric characters and is converted to all uppercase

isDate(this, format)

This function is for fields defined as a date field. It formats data based on the format mask contained in the format parameter. In the example given below, the first digit (1) of the format parameter indicates the date format (MM/DD/YY). The forward slash (/) indicates the separator character (/). The third digit (4) indicates the number of digits in the year. For example, if you enter 021503, this function converts it to 02/15/03.

```
IsDate(this, '1/4')
```

The table below shows only a partial listing of the date formats that can be selected in the Image Editor and the FAP code generated. The FAP code is used for the first digit of the function's format parameter.

DATE FORMAT

Format	Code	Date Formatted As (Examples for January 2, 2013)
MM/DD/YY	1	01/02/13 (default)
DD/MM/YY	2	02/01/13
YY/MM/DD	3	13/01/02
Month D, Yr	4	January 2, 2013
M/D/YY	5	1/2/13
D/M/YY	6	2/1/13
YY/M/D	7	13/1/2

textCounter(field, maxlimit)

This function is for fields defined as multi-line. It will not allow you to enter text exceeding the total length of the multi-line text field.

```
textCounter(field, 120)
```

CSCCImportDPXMLFile Rule

This custom Job Level Two rule is executed by GENDATA to import the XML file created when the HTML form was submitted. If the XML file is not empty and contains matching variable field data with the current document being produced, then the data overrides the data mapped from the extract file.

To implement this rule, follow the steps below:

- 1 Add entry to Document Print's AFGJOB.JDT file

```
< Base Form Set Rules >
:
CSCCImportDPXMLFile;2;POST_PROC;
```

- 2 Add entry to the Document Print's FSISYS.INI file

```
< DATA >
:
DPXMLFile = .\Extract\XMLFile.Dat
```

WIP Purge Program

When Document Print transactions are sent to WIP for batch printing, they are stored in WIP with a status code of “B.” During the next batch cycle run, Document Production will read the WIP database for transactions with this status code, merge them with the batch transactions, and change the status code to “SR,” indicating they have been serviced. A batch purge program called WSPPGW32 is run at the end of the cycle to delete the “SR” status code entries from the WIP database. This prevents the batch cycle from processing these transactions again in the next batch run.

To implement this process, include WSPPGW32.exe at the end of the batch cycle.

Chapter 7: Additional Features of Document Print

This section discusses additional features of Document Print.

Document Print Clean Utility Job

The Document Print clean utility job, DPCleanUtil, deletes temporary folders left behind from processing Document Print transactions.

This section contains information on how to install and configure the initialization file for the utility job.

Configuring the DPCleanUtil.INI File

The DPCleanUtil.INI file contains settings that control the execution of the utility job. The file is located in
\\DocmentPrintService\\AppServer\\DPCleanUtil.

The following is information for all available options to assist in configuring this file.

[BASE] - THIS GROUP SPECIFIES COMMON SETTINGS FOR THE UTILITY

Option	Description	Value
CreateResult-Log	Indicates whether to create a new log file on every run or append to an existing one.	Y - Create log file N - Append to log file
ResultLog	The log file path and file name (Absolute path is required).	For example: \\DocProd\\Mstres\\Docprint\\DPCleanUtil.log
ResultView	Determines whether or not the result is displayed in a pop-up window.	Y - Display result N - Don't display result

Option	Description	Value
RunAtStart	Determines whether or not the utility will run as soon as it is started.	Y - Run at startup N - Do not run at startup
RunAtTime	When setup as a Windows service, this tells the utility job to run at a specified time each day.	No value means the utility will run at startup.

The **TempClean** group lists the different Document Print resource libraries. For example, the base system contains multiple resource libraries for the Document Print system, Exceed, POINT, POINTDMS, etc. There should be one **TempConfig** entry for each environment.

[TEMPCLEAN]

Option	Description	Value
TempConfig	Identifies each environment to run the utility against.	Environment Name - for example: TempConfig = Exceed TempConfig - POINT

The **TempConfig** group defines the settings for each environment specified under the **TempClean** group.

[TEMPCONFIG: ENVIRONMENTNAME]

Option	Description	Value
TempFolder	Location of the temporary folders created for Document Print. Absolute path is required.	For example: DocProd\Mstres\Docprint\Point
CleanFolder-Age	Age of the folder in days. This utility will only delete temporary folders that are older than CleanFolderAge days. Zero means temp directories with current date will be deleted.	Number of Days (for example: 60)
ExcludeFolder	List of folder names that will be excluded from deletion during the deletion process. For example the MSQ_QUEUE and RESOURCE folders are in the same directory as the temp folders are created. These folders should be excluded from deletion.	Comma separated list of folder names. Only folder name is required and not the path. If nothing is specified then all folders in the path specified by the TempFolder option will be deleted.

Option	Description	Value
SaveErrors	Determines if the utility will save Y - any temp folders containing an error in the Document Production by error file (errfile.dat)	Temp folder containing an error will be saved in the folder specified by the ErrorFolder option. N - Temp folder containing an error will not be saved and will be deleted.
ErrorFolder	Path to the error folder containing temporary folders placed there by the utility because the Document Production error file (for example, errfile.dat) in each folder contained an error.	Absolute Path C:\DocProd\Mstrres\Docprint\Exceed\Errors
CleanError-Folder	Indicates whether or not to delete aging folders contained in the path specified by the ErrorFolder option. This option goes with the CleanErrorFolderAge option.	Y - Aged folder will be deleted during the deletion process. N - Aged folder will not be deleted.
CleanError-FolderAge	Age of the folder in days. If CleanErrorFolder = Y, this utility will delete all folders from the path specified in ErrorFolder option that are older than CleanErrorFolderAge days.	Number of Days (For example: 60)

The following is an example of a single environment setup. Note that when you have a single resource library setup for Document Print, the **TempConfig** option must be equal to **Base**.

```
[TempClean]
TempConfig = Base

[TempConfig:Base]
TempFolder = C:\DocProd\mstrres\DocPrint
ExcludeFolder = ERRORS
SaveErrors = N
ErrorFolder = C:\DocProd\mstrres\DocPrint\Exceed\Errors
CleanErrorFolder = N
CleanErrorFolderAge = 20
```

The following is an example of a multi-environment setup:

```
[TempClean]
TempConfig = Exceed
TempConfig = POINT

[TempConfig:Exceed]
TempFolder = C:\DocProd\mstrres\DocPrint\Exceed
CleanFolderAge = 5
ExcludeFolder = Errors
SaveErrors = Y
ErrorFolder = C:\DocProd\mstrres\DocPrint\Exceed\Errors
CleanErrorFolder = Y
CleanErrorFolderAge = 60

[TempConfig:POINT]
TempFolder = C:\DocProd\mstrres\DocPrint\POINT
CleanFolderAge = 5
ExcludeFolder = Errors
SaveErrors = Y
ErrorFolder = C:\DocProd\mstrres\DocPrint\POINT\Errors
CleanErrorFolder = Y
CleanErrorFolderAge = 30
```

Installing DPCleanUtil

Before installing the utility job, make sure you have specified the correct path information under each *Tempfig* in the DPCleanUtil.INI file. Not doing so will cause the DPCleanUtil job to error when it tries to start as a window service.

Steps to install DPCleanUtil as a window service:

- 1 From a Windows command prompt, change directory to the following folder:

```
<drive>:\<target>\DocPrintService\AppServer\DPCleanUtil
```

Where

drive = drive letter where Document Print was installed

target = target path where Document Print was installed

- 2 Open the DPCleanUtil.exe.config file using a text editor, and update the entries under <appSetting> as follows:

```
<appSettings>
  <add key="ServiceName" value="DPCleanUtil" />
  <add key="DisplayName" value="DPCleanUtil" />
</appSettings>
```

Where

ServiceName = the name of the service

DisplayName = the name that is displayed in the service window

- 3 Run the following command:

```
DPCleanUtilInstall
```

- 4 Open DPCleanUtil.InstallLog file to check for any errors.
- 5 If no errors, verify the window service has been started.

Uninstalling DPCleanUtil

Follow the steps below to uninstall the DPCleanUtil utility job:

- 1 From a Windows command prompt, change directory to the following folder:

```
<drive>:\<target>\DocPrintService\AppServer\DPCleanUtil
```

Where

drive = drive letter where Document Print was installed

target = target path where Document Print was installed

- 2 Run the following command:

```
DPCleanUtilUnInstall
```

- 3 Open DPCleanUtil.InstallLog file to check for any errors.

Multiple DPCleanUtil Utility Job running on the Same Machine

If you have multiple Document Print systems installed on the machine, you can install multiple DPCleanUtil jobs with each having a different window service name.

For example, a machine has two versions of the Document Print system installed, one for system test (referred to as “SYS”) and one for unit test (referred to as “IUT”). To install the utility under different window service name, update each configuration file as follows:

For the DPCleanUtil.exe.config file in the SYS environment, change the following entries to:

```
<appSettings>
  <add key="ServiceName" value="DPCleanUtilSYS" />
  <add key="DisplayName" value="DPCleanUtilSYS" />
</appSettings>
```

The service will be installed under the name DPCleanUtilSYS.

For the DPCleanUtil.exe.config file in the IUT environment, change the following entries to:

```
<appSettings>
  <add key="ServiceName" value="DPCleanUtilIUT" />
  <add key="DisplayName" value="DPCleanUtilIUT" />
</appSettings>
```

The service will be installed under the name DPCleanUtilIUT.

To install the utility for each environment, run the following command from a command prompt. You must be in the directory where the DPCleanUtilInstall.EXE is located for that environment.

```
DPCleanUtilInstall
```

To uninstall the utility job, run the following command for that environment:

```
DPCleanUtilUnInstall
```

User Selectable Paragraphs

The *User Selectable Paragraphs* function provides the capability to control the final appearance of a document by allowing the user to dynamically add paragraph(s) to the document when it is rendered in HTML format.

This is accomplished by adding NOTE object(s) to the FAP (image) during the design phase of the document in the Image Editor. NOTE objects provide information for the HTML print driver to interpret. For each NOTE object found on a FAP, the HTML print driver will add a selection box containing a short description for each paragraph that can be added to the image. The user can dynamically add one or more paragraphs to the document by selecting the appropriate description in the selection box.

In addition to being able to select and add paragraphs to a document, a user can select and add logos to the document. For example, the document can be designed to have a selection box containing a list of signature logos to be dynamically added to the document.

The following screen shows an HTML document containing two selection boxes. One contains a list of paragraphs and the other a list of signature logos.

The screenshot shows a web interface for the CSC Personal Benefits Insurance Center. On the left is a red sidebar with a 'CSC' logo and a list of buttons: FORMS LIST, HTML, VIEW GET, SPELL CHECK, REFRESH, PRINT PREVIEW, ARCHIVE, BATCH PRINT, VIEW SAVE, VIEW DELETE, BACK, and EXIT. Below these buttons are 'Zoom' and 'Print' icons, and two dropdown menus labeled 'System' (set to 'Exceed') and 'DIME' (set to 'Y'). The main content area has a 'CSC' logo and the text 'CSC Personal Benefits Insurance Center', 'P. O. Box Ten, Columbia, SC 29202-0010', and '1-800-333-4000'. The date 'May 12, 2003' is displayed. The insured's name and address are 'Debra A Fletcher', '1020 Bel Vista Circle', 'Columbus, GA 31300'. The policy information includes 'RE: Policy Number: MH80000005AT1', 'Insured: Debra A Fletcher', and 'Date of Loss: 02/05/1999'. There are two dropdown menus: one for 'Please select one or more of the following:' with 'No Damage to Property' selected, and another for 'Please select the proper signature:' with 'Kurt W Fuller' selected. A text prompt asks 'Should you have additional information concerning this claim, please contact us.'

CSC
CSC Personal Benefits Insurance Center
P. O. Box Ten, Columbia, SC 29202-0010
1-800-333-4000

May 12, 2003

Debra A Fletcher
1020 Bel Vista Circle
Columbus, GA 31300

RE: Policy Number: MH80000005AT1
Insured: Debra A Fletcher
Date of Loss: 02/05/1999

Please select one or more of the following:
No Damage to Property

Should you have additional information concerning this claim, please contact us.

Please select the proper signature
Kurt W Fuller

After the user has made a selection from each selection box, the document is submitted to rebuild the HTML document. When the HTML document is re-displayed, the selected paragraph and logo are included in the document.

The screenshot displays the CSC Personal Benefits Insurance Center interface. On the left is a red sidebar with a 'FORMS LIST' menu containing options like HTML, WPGET, SPELL CHECK, REFRESH, PRINT PREVIEW, ARCHIVE, BATCH PRINT, WP SAVE, WP DELETE, BACK, and EXIT. Below the menu are 'System' and 'DIME' status indicators. The main content area has the CSC logo and contact information. It shows a date of May 12, 2003, and insured information for Debra A Fletcher. Claim details include Policy Number MH80000005AT1, Insured Debra A Fletcher, and Date of Loss 02/06/1999. A dropdown menu shows 'No Damage to Property' selected. The text states: 'We have received your claim and have had an opportunity to review the loss information and inspect your property. We must respectfully decline payment of your claim due to no damage to the property was noted by our Claims Representative.' It also includes a signature of Kurt W. Fuller as the Claims Representative.

If a different paragraph is desired, the user de-selects the paragraph and selects an alternative. The document is re-submitted and a new HTML document containing the new paragraph will be displayed. This process can continue until the user is satisfied with the document. The document can then be submitted to produce the proof version.

Modify Document Production

Document Production will need to be modified as follows:

- 1 Create the individual FAP files for each section of the document to be produced.
- 2 Create the different subparagraph FAP files that a user could select.
- 3 Add a note to one of the images that will always be triggered, such as a header. In the note, enter specific verbiage in the Subject Line

that will also be added to the INI file. This verbiage is used to identify the note.

- 4 For each identified note, add the FAP name to the text area without an extension followed by a brief description of the paragraph. Each FAP name and description combination should be separated by a CRLF.
- 5 If you want a comment as the first text line such as a note that would direct the user to select one or more of the following paragraphs, enter “none” or any other word that does not also correspond to an actual FAP image.
- 6 Update Form.Dat to include all images. Include the variable paragraphs in the specific order that you want them to be printed, if selected.
- 7 Update Setrcptb.Dat to trigger the variable images based on a new record layout. The new record layout will be identified in the OFFS-FILE if using the flat file ExtrFile layout or will be a closing XML TAG if using an XML file ExtrFile layout.
- 8 Update the INI file to include the < AddDeleteParagraph > section. This new section contains the following information and options:

a NoteSubject = AddDeleteParagraph

Where AddDeleteParagraph would be the verbiage added to the Subject Line in step three above.

b SelectSize = 4

The value “4” specifies the height of the selection box. HTML does not allow a drop-down type of box when you use multiple selections, but you can change the size of the box that is displayed.

c ExtractFormat = FLAT

Set this option to FLAT if you are using a standard flat file ExtrFile, such as produced by the POINT system.

d ExtractFormat = XML

Alternatively, if Document Production is configured to use an XML ExtrFile, then you would use this option instead.

Depending on which type of ExtrFile you selected in step 8c or 8d, you will add one of the following INI sections:

```
< AddDeleteRecordLayout:FLAT >
  RecordID = ADDDEL
  FieldID = IMAGENAME
```

The RecordID is similar to the other RecordID, as detailed in OFFS-FILE. FieldID is also similar to other fields that are contained in OFFS-FILE. This is where Setrcptb.Dat entries are mapped for variable images.

```
< AddDeleteRecordLayout:XML >
  EndingTag = </TRANREC>
  TagToAdd = SUBFORMS
```

The EndingTag identifies an existing TAG in your ExtrFile. You must supply the ending TAG so that the system can add new TAGS (in memory) in front of the closing TAG. The TagToAdd will be the new TAG added. This is where Setrcptb.Dat is configured to trigger variable images.

- 9 Update Document Print AFGJOB file to include the following entry:

```
< Base Form Set Rules >
:
CSCCImportDPXMLFile;2;PRE_PROC;
```

Modify Calling Application

The Calling Application will need to be modified as follows:

A button such as a “REFRESH” button should be added to the screen displaying the HTML document. This button functions exactly the same as the button that produced the initial HTML document. When the user has selected the paragraph to insert into the document, the button is clicked and an HTML request is sent producing the revised HTML document along with an XML file that has all of the variables captured.

Spell Checker

Document Print provides an API that can be called to check the spelling of enterable fields on the HTML Web page prior to rendering the proof version of the document. The API performs the spelling check on the Document Production server by receiving an XML list of all enterable fields from the HTML document. It checks this list against the Document Production dictionary and returns an XML list of spelling corrections for each misspelled word found. The API can also receive an XML list of custom words that the user wants to add to the dictionary.

To implement the spelling check function, Document Production and the calling application must be modified.

Modify Document Production

Document Production must have the following modification applied:

- 1 Add the following entries under the < Data > group in Document Print's FSISYS file:

```
DPXMLFile = .\extract\XMLFile.xml
SPXMLFile = SpellFile.Xml
WDXMLFile = AddWords.Xml
```

Where

DPXMLFile - path and name of the XML input file containing the data gathered from the HTML page. This is the same file that is used when the HTML page is submitted to render the proof version of the document. This filename must match the filename specified for DPXMLFile in step three.

SPXMLFile - name of the XML output file containing the misspelled words.

WDXMLFile - Name of the XML input file containing the words to be added to the dictionary.

- 2 Add the following entries to Document Print's FSISYS file:

```
< SpellChecker >
BaseDictionary = D:\DocProd\DLL\SSCEAM2.CLX
UserDictionary = D:\DocProd\Mstrres\EXCDBASE\Deflib
\USER.CLX
MaxSuggestions = 10
DepthOfSearch = 100
```

Where:

BaseDictionary - Path to the base dictionary.

UserDictionary - Path to the user dictionary for custom words to be added

MaxSuggestions - Maximum number of suggested corrections for misspelled word. Default is 10.

DepthOfSearch - Zero to 100. Zero is fastest with least number of corrections checked. One hundred is slowest, with most words checked. Default is 50.

- 3 Add the following entry to the DOCPRINT.INI file under the ENVIRONMENT group:

```
DPXMLFile = XMLFile.xml
```

Where:

DPXMLFile - Name of the XML input file containing the data gathered from the HTML page. Default name is XMLFILE.dat. This file-name corresponds to the file name specified for DPXMLFile in step one.

Modify Calling Application

The Calling Application will need to provide the necessary user interface code to call the spelling check API. Refer to the ASP code behind the “Spell Check” button on the Document Print Test page (see below) for an example of how to interface with the spelling check function. See Appendix: [Document Print Web Service Interface Definition](#) for more information.

Electronic Signature

Document Print has the ability to capture electronic signatures using third party signature software. The captured signature is embedded into the image of the document allowing the signature to be displayed when the document is printed or viewed in archive.

CSC uses the Topaz Systems Sigplus software (www.topazsystems.com) as the solution for capturing electronic signatures. In the following sections all documentation refers to how Document Print interfaces with the Topaz System software. If a different third party software is used, Document Print may require modifications.

User Workflow

Electronic signatures are captured on the HTML page that is rendered back to the browser. Documents requiring signatures will have an ActiveX control embedded in the document for each signature required.

e-Signature Sample Letter

CSC Personal Benefits Insurance Center
P.O. Box Ten
Columbia, SC 38191-0001
Web: <http://www.cscbenefits.com>

RE: Our Claim Number: [REDACTED]
Our Insured: [REDACTED]
Date of Loss: [REDACTED]
Your Claim Number: [REDACTED]
Your Insured: [REDACTED]

The above referenced case was heard by the arbitration panel on. An award was issued in favor of applicant, CSC Insurance, in the amount of. Arbitration Forum's dictates awards to be paid within thirty (30) days. Our records indicate we have yet to receive payment of this award.

Please forward your immediate payment for the amount of. I have enclosed a copy of the decision for your review.

Applicant's Signature Clear Sign

Date Clear Sign

Agent's Signature Clear Sign

Figure 5. Electronic Signatures, HTML Page

Two buttons, Clear and Sign, are associated with each ActiveX control. The Clear button is used to clear the object if the signature needs to be re-entered. The Sign button is used to set the state of the ActiveX control to ready. A ready state means that the ActiveX control can accept the signature from the signature pad. For each signature required, the appropriate Sign button is clicked and the document is signed using a signature pad. This process continues until all signatures have been captured.

e-Signature Sample Letter

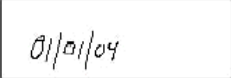
CSC Personal Benefits Insurance Center
P.O. Box Ten
Columbia, SC 36191-0001
Web: <http://www.cscbenefits.com>

RE: Our Claim Number: [REDACTED]
Our Insured: [REDACTED]
Date of Loss: [REDACTED]
Your Claim Number: [REDACTED]
Your Insured: [REDACTED]

The above referenced case was heard by the arbitration panel on. An award was issued in favor of applicant, CSC Insurance, in the amount of. Arbitration Forums dictates awards to be paid within thirty (30) days. Our records indicate we have yet to receive payment of this award.

Please forward your immediate payment for the amount of. I have enclosed a copy of the decision for your review.


Applicant's Signature


Date

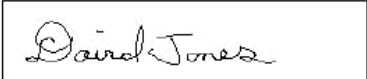

Agent's Signature

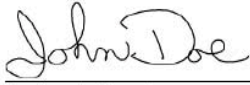
Figure 6. Capturing Signatures

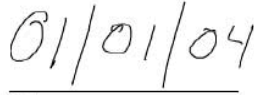
The HTML page is then submitted to render the proof version of the document.

RE: Our Claim Number:
Our Insured:
Date of Loss:
Your Claim Number:
Your Insured:

The above referenced case was heard by the arbitration panel on. An award was issued in favor of applicant, CSC Insurance, in the amount of. Arbitration Forums dictates awards to be paid within thirty (30) days. Our records indicate we have yet to receive payment of this award.

Please forward your immediate payment for the amount of. I have enclosed a copy of the decision for your review.


Applicant's Signature


Date

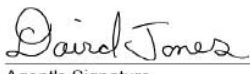

Agent's Signature

Figure 7. Embedded Signatures

Since signatures are embedded in the document, they will be displayed when the document is printed or viewed in the Document Production Archive.

Technical Processing

Capturing electronic signatures and embedding them in a document requires the following steps:

- 1 Design the document to include specially named hidden fields to indicate the document requires signatures.
- 2 The HTML print driver generates the necessary HTML code to capture the signature.
- 3 The text representation of each signature captured on the HTML page is converted to a graphic image (for example, JPG) and stored on the Document Production server.

- 4 Embed each signature's image found on the server into the document so it will be displayed when the proof version of the document is rendered in the Web browser.

Designing the Document

Currently, Skywire's Image Editor does not have an object that can be placed onto a document to indicate the document requires a signature. To rectify this, a specially named hidden field can be inserted and used as a placeholder for the signature. The hidden field has the following requirements.

- To distinguish a signature hidden field from a normal hidden field, the name must be prefixed with a unique name. For example, to capture an applicant's signature, the name of the field could be ESIGNAPPLICANT. The prefix, ESIGN, identifies the field as a signature hidden field. APPLICANT is the real name of the field. An INI setting is used to indicate which prefix name to use for signature hidden fields. The default name is ESIGN.
- The signature hidden field must be positioned where the signature is to appear on the document. The field's coordinates are used for positioning the ActiveX control box and the signature's image on the document.
- If a signature is required, the hidden field's "required" attribute must be selected in the Image Editor. The required attribute is used by the calling application to stop the user from rendering the proof version of the document until all required signatures have been captured.

HTML Print Driver

As the HTML print driver iterates through the FAP objects and encounters a signature hidden field, control is passed to the HTME-SIGN.c module where the following tasks are performed.

- 1 Reads the FSISYS.ini file to get the settings needed to create the OBJECT element to embed the ActiveX control into the HTML document. Following is an example of the entry created:

```
<OBJECT id=APPLICANT name=APPLICANT
classid=clsid:69A40DA3-4D42-11D0-86B0-0000C025864A
CODEBASE="cab/sigplus.ocx" style="position:absolute;
top:8.5in; left:0.5in; width:3.4in; height:0.8in;
```

```
BORDER:SOLID BLACK; border-width:1px;" Required="0"
VIEWASTEXT>
<PARAM NAME="_Version" VALUE="131095">
<PARAM NAME="_ExtentX" VALUE="4842">
<PARAM NAME="_ExtentY" VALUE="1323">
<PARAM NAME="_StockProps" VALUE="0">
</OBJECT>
```

The following property values come from the INI file:

- classid
- CODEBASE
- width
- height
- border-width
- PARAM

The following property values come from the signature hidden field:

- id - name of the hidden field, excluding the prefix
- name - name of the hidden field, excluding the prefix
- left - hidden field's left coordinates
- top - hidden field's vertical coordinates

2 Creates a Clear button.

```
<INPUT id=APPLICANT_clearBtn name=APPLICANT_clearBtn
type=button value="Clear" style="position:absolute;
top:9.483in; left:2.96in;"
onClick="JavaScript:APPLICANT.ClearTablet();">
```

The onClick event handler clears the signature from the ActiveX control.

3 Creates a Sign button.

```
<INPUT id=APPLICANT_signBtn name=APPLICANT_signBtn
type=button value="_Sign " style="position:absolute;
top:9.483in; left:3.50in;"
onClick="turnOnSigPlus(this);">
```

The onClick event handler sets the ActiveX control to a ready state.

Storing Captured Signature on the Document Production Server

After all signatures have been captured, the text representation of the signature is retrieved from each ActiveX control and added to

the XML message that is passed to Document Print when the HTML page is submitted. Following is an example of an XML message containing a signature:

```
<?xml version='1.0' ?>
<formset>
  <form>
    <name>ESIGN</name>
    <images>
      <image>
        <name>ESIGN</name>
        <data>
          <field>Claim Number</field>
          <value></value>
          <field>Date of Loss</field>
          <value></value>
        </data>
      </image>
    </images>
  </form>
  <signatures>
    <signature>
      <name Required="1">APPLICANT</name>
      <value>07004F00F500340100000100020100FFFFEFDFFCFCDFAFBF9F9
FAF8FBF8FDF9FFF801F804F907FA0BFC0FFC12FE14FE120010FF0EFE0
BFD09FB08FA05F902F900F8FEFAFDFFD00FE0300060009010C020D03
10031003100410030F020F000DFD0BFB0BF709F508F107ED05E604E00
0DDFDDEF9E5F6ECF3F3F1F7EFFBF001EE07ED0DF013F417FA17001806
170D151311160D1707150311000DFB0BF50AEF07E704F100FF8E00900
17A0100FF01FF00FEFFFEFEFEFDFFCFCDFBFFF801F703F406F508F809
FA09FE090209060908070A040A020A0009FD07FC06F904F702F601F60
0F8FFFBFEFDFF00FF03FE07FF0800080208040605040703070107FF06
FE06FB05FB03FA03FA01FA00FC00FFFF0100040005000702070306040
60504050304020501030004FE03FC03F903F702F601F700F900FBFFFD
FFFF01FF030005010600060005010301020200040004FF05FF06FE050
00401040103030304020602070008FF08FD09FD09FB07FC05FC03FE00
FFFDFFF</value>
    </signature>
  </signatures>
</formset>
```

When Document Print receives the XML message, the signature text string contained in the VALUE element is converted to an image format (for example, JPG) and then saved to the data folder of the temporary directory created on the Document Production server. Each image will be saved under the name that was passed in the NAME element. For example, the Applicant's signature image will be saved as APPLICANT.jpg. This enables Document Production to embed the signature in the correct hidden field on the document.

Embedding the Signature's Image into the Document

Before a signature image can be embedded into the document, Document Production must be aware of the image's existence. This is accomplished by an image level custom rule called CSCCEnumerateGraphics. The rule searches the temporary data directory for any

files containing the following file extension: TIFF, JPG, and BMP. For each file found, an extract row is created and added to the extract records already in memory. The newly created row will have the following format:

```
1234567890 ROWNAME          FILENAME
```

Where:

1234567890 - Print Request ID

ROWNAME- the signature's image name, excluding the extension (for example, APPLICANT)

FILENAME - the signature's image filename (for example, APPLICANT.jpg) Field always starts in position 71

After all extract rows have been created by CSCCEnumerateGraphics, another image level custom rule, CSCCInlineBitmap, is called to actually embed the image.

CSCCInlineBitmap is a modified version of Skywire's CUSInlineBitmap base rule. It requires a search mask to search for extract records created by CSCCEnumerateGraphics. For each record found, the image filename and type are extracted from the record, the file is located and retrieved from the data directory, and the image is embedded in the document by setting the embed bitmap flag.

The rule requires six parameters:

```
;CSCCInlineBitmap;offset1,value,offset2,len,type,fldnm
```

Where:

offset1 - record name offset

value - record name

offset2 - Image filename offset

len - image filename length

type - image file type

fldnm - name of the hidden field to which the image belongs

For example:

```
;CSCCInlineBitmap;13,APPLICANT,71,13,9,ESIGNAPPLICANT
```

This will search for the record APPLICANT at offset 13. It retrieves an image filename at offset 71 for a length of 13 (for example, APPLICANT.JPG). The image file type is 9 (meaning it is a JPG) and the hidden fieldname for which the image belongs to is ESIGNAPPLICANT.

Implementation

To implement electronic signature, both Document Production and the calling application will need to be updated.

Modifying Document Production

- 1 Add the image level rules listed below to the image containing the hidden field(s) used for signature(s). It must be in the order shown.

```
;CSCCEnumerateGraphics;;
;CSCCInlineBitmap;13,APPLICANT,71,13,9,ESIGNAPPLICANT;
;CSCCInlineBitmap;13,AGENT,71,9,9,ESIGNAGENT;
```

A CSCCInlineBitmap entry is required for each signature hidden field on the image.

- 2 Update FSISYS.ini or the product-specific INI file (for example, EXCDBASE.ini) to include the entries below. ESIGN_PARAMS controls how the HTML print driver builds the tags to display the ActiveX control when the document is rendered in HTML format:

```
< ESIGN_PARAMS >
Prefix = ESIGN
Caption = No
BuildXML = No
LoadButton = No
ClassID = clsid:69A40DA3-4D42-11D0-86B0-0000C025864A
CodeBase = cab/sigplus.ocx
Width = 8300
Height = 2000
BorderWidth = 1
PARAM = NAME="_Version" VALUE="131095"
PARAM = NAME="_ExtentX" VALUE="4842"
PARAM = NAME="_ExtentY" VALUE="1323"
PARAM = NAME="_StockProps" VALUE="0"
```

Where:

Prefix - prefix name to use for e-signature hidden fields. Default: ESIGN.

Caption - caption the ActiveX control (Yes/No). Default: No. If “Yes,” the ActiveX control is caption with the name of the hidden field, excluding the prefix.

BuildXML - builds an XML message containing a list of signature hidden fields found on the document (Yes/No). Message is passed back with the HTML document. Not used by the base system. Default: No.

LoadButton - creates a “Load” button for each ActiveX control (Yes/No). Used for testing purposes to load a pre-defined signature into the control when a signature pad is not available. Default: No.

ClassID - class ID for the ActiveX control.

CodeBase - location of the base directory in which the browser will look for data and other implementation files.

Width - width (in FAP units) of the ActiveX control box. Default: 8500.

Height - height (in FAP units) of the ActiveX control box. Default: 2400.

BorderWidth - border width of the Active-X control box. Default: 1.

PARAM - additional parameters needed by the Active-X control.

- 3 Update the DOCPRINT.ini file to include the entries below. SIGPLUS_SETTINGS controls some property values for the Topaz's Sigplus software.

```
[ SIGPLUS_SETTINGS ]
ImageXSize= 700
ImageYSize= 300
JustifyMode= 4
ImagePenWidth= 12
ImageFileFormat = 4
SigCompressionMode = 2
```

Where:

ImageXSize - width of resulting image in pixels.

ImageYSize - height of resulting image in pixels.

JustifyMode - sets justification mode.

ImagePenWidth - pen width (in pixels).

ImageFileFormat - image file type to produce.

4 - JPG

6 - Compressed TIF

SigCompressionMode - Compression Mode

Note: Refer to the Topaz System's SigPlus documentation for additional information on these property values.

Forms and Variable Fields List Function

The process of generating an on-demand document requires a user to select a document from a candidate list. A table that is stored and maintained on the Web application is usually responsible for generating this list. This table is known as the Print Candidate List in the Exceed application. However, Document Production also has a table, called the Form Set Definition Table (FORM.dat), which contains a list of all forms available for printing. Web applications not wanting to maintain a separate table can call the forms list API to generate a print candidate's list from the FORM.dat file.

There are four different commands that can be passed to the forms list interface: ALL, FORM, SEARCH, and VAR. Following is a description of each command:

ALL

This command generates an XML list of all forms defined in the form set definition table.

FORM

This command generates an XML list of forms based on a search on the form's description field using a phrase value. Using the FORM.dat file (shown below as an example), a list of forms having the phrase "Claims" in the form's description can be generated by passing a command of "FORM" and a search mask of "Claims" to the interface. The search mask is not case sensitive.

```
;BCMS;BILLING;BCMS999;Billing
Letter;N;;BCMS999|D<PAY(0)>;
;CWS;CWS;CLMDENLT;Claims Denial
Letter;N;;CLMDENLT|D<INS(0)>;
;CWS;CWS;CLMNOTLS;Notice of Loss;N;;CLMNOTLS|D<INS(0)>;
;CWS;CWS;CLMREPLT;Claims Rep
Letter;N;;CLMREPLT|D<INS(0)>;
;CWS;CWS;ESIGN;ESignature Letter;N;;ESIGN|D<INS(0)>;
;DWS;DWS;DBILCKNP;Billing Check;N;;DBILCKNP|D<PAYOR(0)>;
;DWS;DWS;DCLMCKNP;Claims Check;N;;DCLMCKNP|D<PAYOR(0)>;
```

The list generated would match the following entries:

```
;CWS;CWS;CLMDENLT;Claims Denial
Letter;N;;CLMDENLT|D<INS(0)>;
;CWS;CWS;CLMREPLT;Claims Rep
Letter;N;;CLMREPLT|D<INS(0)>;
;DWS;DWS;DCLMCKNP;Claims Check;N;;DCLMCKNP|D<PAYOR(0)>;
```

SEARCH

This command generates a list of forms based on a search on the form's description using a search mask value. The search mask value is in the format of offset,data. For example, if you want to return a list of all forms containing a state code of SC in position 1 in the form's description field, a command of "SEARCH" with a search mask of "1,SC" would be passed to the interface.

```
;CWS;CWS;CLMDENLT;SC-Claims Denial
Ltr;N;;CLMDENLT|D<INS(0)>;
;CWS;CWS;CLMNOTLS;Notice of Loss;N;;CLMNOTLS|D<INS(0)>;
;CWS;CWS;CLMREPLT;SC-Claims Rep
Letter;N;;CLMREPLT|D<INS(0)>;
;CWS;CWS;ESIGN;ESignature Letter;N;;ESIGN|D<INS(0)>;
;DWS;DWS;DBILCKNP;SC-Billing
Check;N;;DBILCKNP|D<PAYOR(0)>;
;DWS;DWS;DCLMCKNP;Claims Check;N;;DCLMCKNP|D<PAYOR(0)>;
```

The list generated would match the following entries:

```
;CWS;CWS;CLMDENLT;SC-Claims Denial
Ltr;N;;CLMDENLT|D<INS(0)>;
;CWS;CWS;CLMREPLT;SC-Claims Rep
Letter;N;;CLMREPLT|D<INS(0)>;
;DWS;DWS;DBILCKNP;SC-Billing
Check;N;;DBILCKNP|D<PAYOR(0)>;
```

VAR

This command generates a list of all variable fields that make up a specific form. For example, to return a list of all variable fields for the "Notice of Loss" form, a command of "VAR" with a form name of "CLMNOTLS" would be passed to the interface. This would cause the FORM.dat to be searched for a match on the form name "CLMDEN-LTR." For each image that makes up the form, the variable fields contained in the image's DDT file would be added to the list that is returned.

To further filter the list generated by any of the above commands, the KEY1 and/or KEY2 value defined in the FORM.dat file can be passed. For example, if a KEY1 value of "CWS" was passed with an "ALL" command, all forms having a KEY1 value of "CWS" will be returned in the list.

Implementing the Forms and Variable Fields List Function

To implement this function, Document Production and the calling application will need to be modified.

Modify Document Production

- 1 Update the FSISYS.ini file to include the following entries under the <DATA> group:

```
XmlFormName = FormList.Xml
XmlVarName = VarList.Xml
```

Where:

XmlFormName - Output filename when generating an XML list for the commands ALL, FORM, and SEARCH.

XmlVarName - Output filename when generating an XML list for the VAR command.

- 2 If the VAR command will be used, add the following control groups to the INI file:

```
< FormList_UseParams >
  ConCat = TRUE
  TblLook = TRUE
  SetAddr = TRUE
```

This INI group lists the rules that use RuleParm and not the Source Field Name to map from. This will instruct the program to parse the RuleParm for each rule listed to get a list of variable fields.

```
< FormList_SkipRules >
  NoopFunc = TRUE
  KickToWip = TRUE
  PowType = TRUE
  FfSysDte = TRUE
  NoMap = TRUE
  mk_hard = TRUE
```

This INI option instructs the program to skip all rules listed here because these rules do not map or produce output from the extract file.

Modify Calling Application

The calling application will need to provide the user interface code to call and process the result from this API. Refer to the ASP code behind the “Forms List” button on the Document Print Test page for

an example. Also refer to Appendix: [Document Print Web Service Interface Definition](#) for the documented interface specification.

Exclude HTML Forms with No Manual Entry Fields from Rendering in the Browser

Document Print can limit the number of HTML forms that are rendered in the browser by excluding forms that do not have the KickToWip or PowType field rule, and the operator required flag (Flag3) equal to “Y.” For example, a document such as a homeowner declaration could contain several forms that have nothing but static text. To render these HTML forms in the browser will not only impact performance but also cause the user to page through forms that have no manual entry fields.

Implementation

To have the HTML print driver exclude forms with no manual entry fields, follow these steps:

- 1 Add the following INI options to the group <PrtType:HTM> in the Document Print FSISYS file.

```
< PrtType:HTM >
    LimitFormsByPowType = Yes
    NoFormsGenMsg = User defined message
```

In the event that no forms were generated, the message defined for the INI option, NoFormsGenMsg, will be displayed.

- 2 Fields requiring manual entry, add the following field level rule in the DDT. Also, set the operator required flag to “Y.”

```
;KickToWip;;N;N;Y;N;
or
```

```
;PowType;;N;N;Y;N;
```

To override the flag set for the HTML print driver that excludes forms in IAW, follow these steps:

- 1 Add the following INI option to the group <WipData> in the FSISYS file.

```
< WipData >
    KWOverrideField = NameOfField
```

If the field indicated by NameOfField exists and contains data in WIP, the flag for the HTML print driver to exclude forms is overridden.

- 2 Define NameOfField in your WIP.DFD file.
- 3 Add EXT2GVM or CSCCEXT2GVM2 rule to the Document Print AFGJOB to populate the NameOfField global field from the extract file.

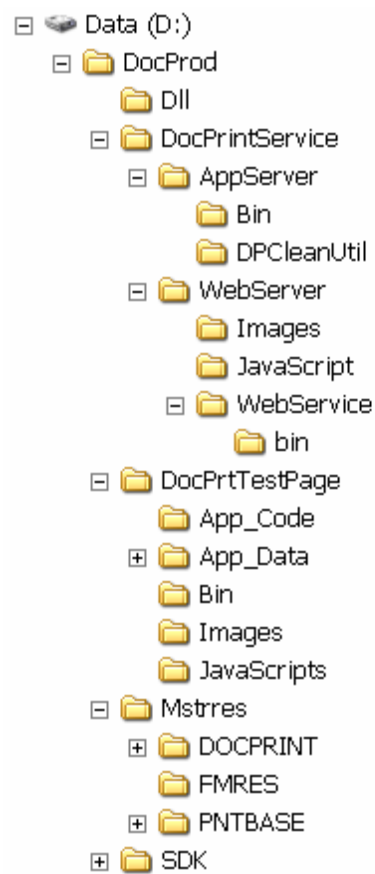
If the field indicated by NameOfField exists and contains data in WIP, the flag for the HTML print driver that excludes forms is overridden. By doing so, only those policies with this special field populated will be able to display all forms, even though the INI file says to exclude the forms.

Chapter 8: How to Manually Install Document Print

The installation CD automatically configures Document Print during the installation. However, you may need to manually configure an environment for Document Print. For example, you may have a Document Print environment installed on one server but you need to copy this environment and configure it on a different server.

This chapter discusses how to manually create a Document Print environment from an existing one. It includes instructions on how to configure Document Print as a 1 server configuration and as a 2 server configuration.

The Document Print environment shown below will be used for illustration purposes.



The following assumptions are made:

- The existing environment is configured as a 1 server configuration (the Web and application components are on the same machine).
- The new environment will be created on a server different from the existing one.

Install Document Print as a One Server Configuration

To create a 1 server configuration environment from an existing environment follow the steps below:

- 1 Copy the folder \DocProd to the target machine.
- 2 In IIS, create a new application pool for the Document Print virtual directories to reside in.
 - a Right click on the **Application Pools** node from the tree view.
 - b Select **New > Application Pool** from the context menu.
 - c Enter a name for the application pool (for example, DocPrintWebService).
 - d Click **OK**.

- 3 Create a virtual directory for the test page:

Virtual Directory: **DocPrtTestPage**
Path: <drive>:\<target>\DocPrtTestPage
Application Name: **DocPrtTestPage**
Permission: **Read, Scripts only**
Application Pool: **Name of pool created in step 2**

Where:

drive = drive letter of where Document Print was installed
target = target path where Document Print was installed

- 4 Create a virtual directory for the Web Service:

Virtual Directory: **DocPrintService**
Path: <drive>:\<target>\DocPrintService\WebServer
Application Name: **DocPrintService**
Permission: **Read, Scripts only**
Application Pool: **Name of pool created in step 2**

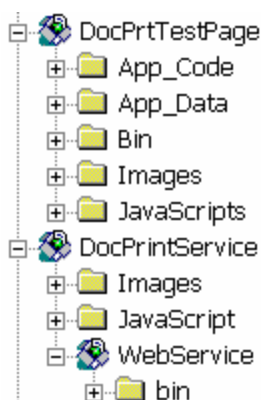
Where:

drive = drive letter of where Document Print was installed
target = target path where Document Print was installed

- 5 Make the folder **WebService**, under the **DocPrintService** virtual directory, an application.

Virtual Directory: **WebService**
 Application Name: **WebService**
 Permission: **Read, Scripts only**
 Application Pool: **Name of pool created in step 2**

The Web site should look like the following:



- 6 If ASP.NET 2.0 and a prior version co-exist on the server, the two virtual directories above must have its ASP.NET version set to 2.0.
- In IIS, right click on the virtual directory name and select **Properties**.
 - Select the **ASP.NET** tab.
 - In the selection box for ASP.NET version, select **2.0.50727**.
 - Click **OK**

- 7 Under <appSettings> in \DocPrtTestPage\web.config, change key value for **DocPrintURL** to point to the proper URL:

`http:// IPAddress /DocPrintService/WebService/
DocPrintService.asmx`

- 8 Under <appSettings> in \DocPrintService\WebServer\WebService\web.config change path information for key value **DPINILOC**

`value="<drive>:\<target>\DocPrintService\AppServer\bin"`

Where:

drive = drive letter of where Document Print was installed

target = target path where Document Print was installed

- 9 Edit **DOCPRINT.INI**, located in \DocPrintService\Appserver\Bin, to reflect the proper path and settings for the environment.
- 10 In \Mstrres\DocPrint\POINT\FSISYS.INI, update the URLs under the group <PRTTYPER:HTM>.

GifPath = http:\\<ipaddress>\DocPrintService\Images\

JSPath =

http:\\<ipaddress>\DocPrintService\JavaScript\htmscrip.js

ScriptFile =

http:\\<ipaddress>\DocPrintService\JavaScript\buildxml.js

ScriptFile =

http:\\<ipaddress>\DocPrintService\JavaScript\verify.js

ScriptFile =

http:\\<ipaddress>\DocPrintService\JavaScript\HTMESIGN.js

- 11 Test the installation using the Document Print Test Page.

Install Document Print as a Two Server Configuration

To create a 2 server configuration environment from an existing environment, follow the steps below.

Application Server Installation

- 1 Copy the following directories:

DocProd\Dll*. * to DocProd\Dll*. *

DocProd\DocPrintService\AppServer*. * to
DocProd\DocPrintService\AppServer*. *

DocProd\Mstres*. * to DocProd\Mstres*. *

DocProd\SDK*. * to DocProd\SDK*. *

- 2 Edit **DOCPRINT.INI** located in
DocProd\DocPrintService\Appserver\Bin, to reflect the proper path
and settings for this environment.
- 3 In DocProd\Mstres\DocPrint\POINT\FSISYS.INI, update the URLs
under the group <PRTTYPE:HTM>.

GifPath = http:\\<ipaddress>\DocPrintService\Images\

JSPath =
http:\\<ipaddress>\DocPrintService\JavaScript\htmscrip.js

ScriptFile =
http:\\<ipaddress>\DocPrintService\JavaScript\buildxml.js

ScriptFile =
http:\\<ipaddress>\DocPrintService\JavaScript\verify.js

ScriptFile =
http:\\<ipaddress>\DocPrintService\JavaScript\HTMESIGN.js

- 4 Specify a **Port** value for the remote listener to listen on under
<appSettings> in.

DocProd\DocPrintService\Appserver\Bin\
DocPrintListener.exe.config

- 5 From a command prompt, execute the following BAT file to install the listener:

\DocPrintService\AppServer\DocPrintListenerInstall.bat

Web Server Installation

- 1 Copy the following directories:

DocProd\DocPrintService\WebServer*. * to
DocProd\DocPrintService\WebServer*. * to
DocProd\DocPrtTestPage*. * to DocProd\DocPrtTestPage*. *

- 2 Do steps 2 - 6, specified under the 1 server configuration.
- 3 Under <appSettings>, in \DocPrtTestPage\web.config, change **DocPrintURL** to point to the correct URL.

value=http:// *IPAddress* /DocPrintService/WebService/
DocPrintService.asmx

Where IPAddress is the IPAddress of the web server

- 4 Under <appSettings>, in
\DocPrintService\WebServer\WebService\web.config, change path
information for key value **DPINILOC**

value="<drive>:\<target>\DocPrintService\AppServer\bin"

Note: <drive> is the actual drive on the application server and not a mapped drive.

- 5 Edit \DocPrintService\AppServer\Bin\DocPrintListenerInstall.bat as follows:

String	Replace
<add key="Port" value="8080" />	Port Value. Must be same value as specified on the application server
<add key="remoteserver" value="127.0.0.1"/>	IP address of the App server
<add key="remotingenabled" value="false"/>	Set value to "true

Chapter 9: Document Change Log

Date	Release for which change sub made	Heading and heading where change made	Description	Initials

Appendix A: Document Print Web Service Interface Definition

This appendix defines the interfaces for the Document Print application. Applications wanting to interface with Document Print can use the information provided here to build the necessary code to communicate with each WebMethod.

Web Service

DocPrintService.asmx

Class

DocPrintService

WebMethod

```
Public Function callDocPrtFormsList(ByVal strCommand As String, _  
                                   ByVal strUniqueID As String, _  
                                   ByVal strGroup1 As String, _  
                                   ByVal strGroup2 As String, _  
                                   ByVal strSrchMask As String, _  
                                   ByVal strEnvrnmnt As String) _  
    As String
```

Description

This method generates a list of forms using the Document Production Form Set Definition file. Also, it can generate a list of variable fields on a form.

Parameters

strCommand (I) - indicates the type of list to return. Field is required and must be one of the following:

ALL - generates a list of all forms contained in the FORM.DAT file.

FORM - generates a list of forms based on a search on the FORM.DAT form's description field using the phrase value specified in the SrchMask parameter.

SEARCH - generates a list of forms based on a search on the FORM.DAT form's description field using the search mask value (in the format of offset,data), specified in the SrchMask parameter.

VAR - generates a list of fields for all images that makes up the form specified in the SrchMask parameter.

strUniqueID (I) - a unique identifier number used to create the temporary environment. Field is required.

strGroup1 (I) - an optional field used to restrict the list generated by the type of command passed in the Command parameter. Field corresponds to the KEY1 field in the FORM.DAT file.

strGroup2 (I) - an optional field used to restrict the list generated by the type of command passed in the Command parameter. Field corresponds to the KEY2 field in the FORM.DAT file.

strSrchMask (I) - contains a query value and is required for the following commands: FORM, SEARCH, and VAR. The query value is not case sensitive. When used with the command:

FORM - the query value is a phrase. For example, to generate a list of forms that has "Claims Letter" in the form's description field, pass the phrase "Claims Letter" as the query value (excluding quotes).

SEARCH - the query value is a search mask. For example, to generate a list of forms that has a state code of SC in position 20 in the form's description field, pass 20,SC as the query value.

VAR - the query value is a form name that matches the form name field in the FORM.DAT file. For example, to generate a list of all variable fields belonging to the form, CLMDENLT, pass this name as the query value.

strEnvrnmnt (I) - reads DOCPRINT.INI to determine which Document Production environment to execute in a multi-environment setup. Field is optional.

Return

An XML file is returned containing a list of forms or variable field on a form.

For a list of forms, the XML string will be returned in the following layout:

```
<?xml version="1.0" ?>
<DocPrintForms>
  <Group Name1="xxxxxx" Name2="xxxxxx">
    <Form Name="xxxxxx" Desc="xxxxxx"/>
    <Form Name="xxxxxx" Desc="xxxxxx"/>
  </Group>
  <Group Name1="yyyyyy" Name2="yyyyyy">
    <Form Name="yyyyyy" Desc="yyyyyy"/>
  </Group>
</DocPrintForms>
```

Where:

<Group> Name1 attribute = KEY1 value in the FORM.DAT.
Name2 attribute = KEY2 value in FORM.DAT.

<Form> Name attribute = form name field in FORM.DAT.
Desc attribute = form description field in FORM.DAT.

An example of this XML document is given below:

```
<?xml version="1.0"?>
<DocPrintForms>
  <Group Name1="CWS" Name2="CWS">
```

```

    <Form Name="CLMDENLT" Desc="Claims Denial Letter"/>
    <Form Name="CLMNOTLS" Desc="Notice of Loss"/>
    <Form Name="CL102" Desc="Public Safety Loss Rpt"/>
  </Group>
  <Group Name1="DWS" Name2="DWS">
    <Form Name="DBILCKPP" Desc="Billing Check Pre-Printed"/>
    <Form Name="DBILCKNP" Desc="Billing Check"/>
    <Form Name="DCLMCKPP" Desc="Claims Check Pre-Printed"/>
    <Form Name="DCLMCKNP" Desc="Claims Check"/>
  </Group>
</DocPrintForms>

```

For a list of variables, the XML document will be returned in the following layout:

```

<?xml version="1.0" ?>
<DocPrintVariables>
  <Image>
    <Name></Name>
    <Fields>
      <Name></Name>
      <Name></Name>
    </Fields>
  </Image>
  <Image>
    <Name></Name>
    <Fields>
      <Name></Name>
    </Fields>
  </Image>
</DocPrintVariables>

```

Where:

<Image>< Name> - name of the image.
 <Image><Fields><Name> - name of the field.

An example of this XML document is given below:

```

<?xml version="1.0" ?>
<DocPrintVariables>
  <Image>
    <Name>CLMDENLT</Name>
    <Fields>
      <Name>CLMLOSSDT</Name>
      <Name>CLTLNGNM</Name>
      <Name>CVRPOLNBR</Name>
      <Name>CVRPOLSYMCD</Name>
    </Fields>
  </Image>
</DocPrintVariables>

```

</Image>
</DocPrintVariables>

WebMethod

```
Public Function callDocPrtSpellCheck (ByVal strCommand As String,
                                     ByVal strUniqueID As String, _
                                     ByVal strEnvrnmnt As String, _
                                     ByVal strXMLDataIn As String, _
                                     ByRef strRetXmlList As String) _
    As Integer
```

Description

This method checks the spelling of input fields on the HTML page against the Document Production dictionary. It also allows custom words to be added to the dictionary.

Parameters

strCommand (I) - indicates whether to check for spelling errors or add word(s) to the dictionary. Field is required and must have one of the following values:

ADD - adds the word(s) contained in the XML document that is passed in to the dictionary.

CHECK - checks the spelling of each word contained in the XML document that is passed in.

strUniqueID (I) - a unique identifier number used to create a temporary environment. Field is required.

strEnvrnmnt (I) - reads DOCPRINT.ini to determine which Document Production environment to execute in a multi-environment setup. Field is optional.

strXMLDataIn (I) - an XML list of all input fields gathered from the HTML page to be checked for spelling errors or an XML list of words to be added to the user dictionary.

For the CHECK command, the layout of this XML document must be in the same layout as the XML list passed when an HTML page is submitted. For the ADD command, the layout of this XML document must be in the following format:

```
<?xml version="1.0" ?>
<AddWords>
  <word></word>
  <word></word>
</AddWords>
```

Where:

<AddWords><word> contains the word to be added to the dictionary.

An example of this XML document is given below:

```
<?xml version="1.0" ?>
<AddWords>
  <word>CSC</word>
  <word>Docucorp</word>
</AddWords>
```

strRetXmlList (O) - an XML list of misspelled words, along with spelling suggestion, returned for a CHECK command.

The XML document is returned in the following layout:

```
<?xml version="1.0" ?>
<DocPrintSpellCheck>
  <errorword>
    <form></form>
    <image></image>
    <field></field>
    <value></value>
    <word></word>
    <suggest></suggest>
    <suggest></suggest>
    <suggest></suggest>
  </errorword>
  <errorword>
    <form></form>
    <image></image>
    <field></field>
    <value></value>
    <word></word>
    <suggest></suggest>
    <suggest></suggest>
    <suggest></suggest>
  </errorword>
</DocPrintSpellCheck>
```

Where:

<errorword><form> represents the name of the form the error was found on. This corresponds to the NAME attribute of the HTML element FORM.

<errorword><image> represents the name of the image the error was found on. This corresponds to the ID attribute of the HTML element DIV.

<errorword><field> represents the name of the field the error was found on. This corresponds to the NAME attribute of the HTML element INPUT or TEXTAREA.

<errorword><value> represents the value of the field the misspelled word was found in. This corresponds to the VALUE attribute of the HTML element INPUT or TEXTAREA.

<errorword><word> represents the misspelled word. This corresponds to the value in the VALUE attribute of the HTML element INPUT or TEXTAREA.

<errorword><suggest> represents a spelling suggestion for the misspelled word.

An example of this XML document is given below:

```
<?xml version="1.0" ?>
<DocPrintSpellCheck>
  <errorword>
    <form>CLMDENLT</form>
    <image>CLMDENLT</image>
    <field>OTHER REASON</field>
    <value>This is a mixspelled sentence</value>
    <word>mixspelled</word>
    <suggest>misspelled</suggest>
    <suggest>dispelled</suggest>
    <suggest>miscalled</suggest>
    <suggest>misspell</suggest>
  </errorword>
</DocPrintSpellCheck>
```

Return

0 (Success)

-1 (Failure)

WebMethod

Public Function callDocumentProduction (ByVal strReqType As String, _
 ByVal strUniqueID As String, _
 ByVal strEnvrnmnt As String, _
 ByRef strRetMIMEType As String) _
 As Integer

Description

This method processes a transaction to produce a print file (HTML, PDF, AFP, etc.) or send a document to the Document Production WIP or ARCHIVE database.

Parameters

strReqType (I) - indicates what process to be performed by Document Production. The following are the different request types that can be passed. Field is required.

HTML - returns document in HTML format.

PRINT - returns document in format defined in the FSISYS.INI file (for example, PDF, AFP, etc.).

PDF - returns document in PDF format.

AFP - returns document in AFP format.

BATCH - sends document to WIP for batch printing

ARCHIVE - sends document to Archive.

EXIT - deletes temporary environment.

Below are POINT IN specific request types:

PNT_WIP - Sends transaction to WIP with a "W" status code

BATCH_DEC - Produce document and send transaction to WIP for batch print

ARCHIVE_DEC - Produce document and send transaction to archive

strUniqueID (I) - a unique identifier number used to create the temporary environment. Field is required.

strEnvrnmnt (I) - reads DOCPRINT.ini to determine which Document Production environment to execute in a multi-environment setup. Field is optional.

strRetMIMETYPE (IO) - MIME type of the proof version document returned to the calling application. It is used to set the content type on the calling application side. It indicates which browser plug-in to initiate (for example, PDF - Acrobat Reader, AFP - IBM AFP plug-in). Value comes from PRTTYPE, under the CONTROLS group, in the DOCPRINT.INI file.

DIME Attachments

Input Attachment 1 - an XML document consisting of all input fields gathered from the HTML form. Used by GENDATA to update the form with modified data before the proof version of the document is produced and sent back to the browser. The DIME ID must equal CHG-DATA so the file can be referenced in the collection.

This XML document must be in the following layout:

```
<?xml version='1.0' ?>
<formset>
  <form>
    <name></name>
    <images>
      <image>
        <name></name>
        <data>
          <field></field>
          <value></value>
        </data>
      </image>
    </images>
  </form>
</formset>
```

Where:

<form><name> represents the name of the form. This corresponds to the NAME attribute of the HTML element FORM.

<image><name> represents the name of each image on the form. This corresponds to the ID attribute of the HTML element DIV.

<image><data><field> represents the name of each variable field on the image. This corresponds to the NAME attribute of the HTML element INPUT or the TEXTAREA.

<image><data><value> represents the data value from the variable field. This corresponds to the VALUE attribute of the HTML element INPUT or the TEXTAREA.

An example of this XML document is given below:

```
<?xml version="1.0" ?>
  <formset>
    <form>
      <name>CLMDENLT</name>
      <images>
        <image>
          <name>LTRHEAD</name>
          <data />
        </image>
        <image>
          <name>CLMDENLT</name>
          <data>
            <field>REASON 1</field>
            <value />
            <field>REASON 2</field>
            <value />
            <field>REASON 3</field>
            <value>X<value />
            <field>OTHER REASON</field>
            <value>No coverage</value>
          </data>
        </image>
      </images>
    </form>
  </formset>
```

Input Attachment 2 - an extract file consisting of the data gathered from the calling application system. Data must be passed as a byte array because some extract files can contain binary data. The DIME ID must equal EXTRDATA so the file can be referenced in the collection. Field is required on initial call but optional on subsequent calls. For example, when an initial call is made to produce an HTML document, the extract data must be passed. On the next call to produce the PDF version, extract data is not required because Document Print will use the extract data stored in the temporary directory from the HTML call.

Output Attachment 1 - the document print stream (HTML, PDF, AFP, etc.). The DIME ID will equal PRCDATA so the file can be referenced in the collection.

Return

0 (Success)

-1 (Failure)

WebMethod

```
IAWWIPSave(ByVal strUniqueID As String, _  
            ByVal strEffDate As String, _  
            ByVal strEnvrnmnt As String) _  
            As Integer
```

Description

This method saves a document to the Document Production WIP database.

Parameters

strUniqueID (I) - a unique identifier number used to create the temporary environment. Field is required.

strEffDate (I) - transaction effective date. Field is for future use.

strEnvrnmnt (I) - reads DOCPRINT.ini to determine which Document Production environment to execute in a multi-environment setup. Field is optional.

DIME Attachment

Input Attachment - XML document consisting of all input fields gathered from the HTML form. The data is used to update the form with any change values and is in the same format as the CHGDATA for WebMethod callDocumentProduction. The DIME ID must equal CHGDATA so the file can be referenced in the collection. Field is optional.

Return

0 (Success)

-1 (Failure)

WebMethod

```
Public Function IAWWIPGet(ByVal strUniqueID As String, _  
                          ByVal strEffDate As String, _  
                          ByVal strRecipName As String, _  
                          ByVal strEnvrnmnt As String) _  
                          As Integer
```

Description

This method retrieves an existing document in the Document Production WIP database.

Parameters

strUniqueID (I) - a unique identifier number used to create the temporary environment. Field is required.

strEffDate (I) - the transaction effective date. Field is for future use.

strRecipName (I) - the recipient print data to return. Field is for future use.

strEnvrnmnt (I) - reads DOCPRINT.ini to determine which Document Production environment to execute in a multi-environment setup. Field is optional.

DIME Attachment

Input Attachment 1 - XML document consisting of all input fields gathered from the HTML form. The data is used to update the form with any change values and is in the same format as the CHGDATA for WebMethod call DocumentProduction. The DIME ID must equal CHGDATA so the file can be referenced in the collection. Field is optional.

Input Attachment 2 - an extract file consisting of the data gathered from the calling application system. Data must be passed as a byte array because some extract file can contain binary data. The DIME ID must equal EXTRDATA so the file can be referenced in the collection.

Output Attachment - an HTML print stream. The DIME ID will equal PRTRDATA so the file can be referenced in the collection.

Return

0 (Success)

-1 (Failure)

WebMethod

Public Function IAWWIPDelete(ByVal strUniqueID As String, _
ByVal strEnvrnmnt As String) _
As Integer

Description

This method deletes an existing document in the Document Production WIP database.

Parameters

strUniqueID (I) - a unique identifier number used to create the temporary environment. Field is required.

strEnvrnmnt (I) - reads DOCPRINT.ini to determine which Document Production environment to execute in a multi-environment setup. Field is optional.

Return

0 (Success)

-1 (Failure)

Appendix B: Media Management Web Method Interface Definition

This appendix defines the new Web methods added to the Document Print Web service for Media Management. Applications wanting to interface with these Web methods can use the information provided here to build the necessary code for calling each Web Method.

Web Service

DocPrintService.asmx

Class

DocPrintService

Web Method

```
Public Function MMWIPSave(ByVal XmlRequestMsg As String, _  
                          ByRef XmlResponseMsg As String) _  
                          As Integer
```

Description

This method is used to save a document to the Document Production WIP database

Parameters: I - Input O - Output

XmlRequestMsg (I) - an XML message consisting of parameter values to be passed to the web method. Required format for the XML message is as follow: